Instructions for Implementing RAG Systems with VolleyBall Dataset

📚 Overview

This document provides comprehensive instructions for implementing three progressive RAG (Retrieval-Augmented Generation) systems using the **VolleyBall dataset**. Each implementation builds upon the concepts from the Python documentation examples but introduces domain-specific challenges and enhanced capabilities for handling sports data, rules, and tabular information.

Dataset Description

Location: `UnstructuredDataVolleyBall/`

The VolleyBall dataset contains diverse document types related to volleyball:

```
UnstructuredDataVolleyBall/
├── CSVFiles/
│   └── vb_matches.csv              # Match statistics and game data
├── HTMLFiles/
│   ├── Adult Sand Volleyball Rules - Canon City Area Metropolitan Recreation &
│   └── Beach Volleyball League Rules — Volley Life.html
├── PDFFiles/*
└── TextFiles/
    ├── Junior_High_Lesson_Plans.txt
    └── USL_Grass_Volleyball_Rules.txt
```

Document Types:

- **CSV Files**: Tabular match data with statistics, scores, teams, dates
- **HTML Files**: Official volleyball rules and regulations (2 files)
- **Text Files**: Educational content - lesson plans and grass volleyball rules (2 files)

🎯 Learning Objectives

By completing these three notebooks, you will:

1. **Adapt RAG systems** to domain-specific datasets (sports/volleyball)
2. **Handle tabular data** (CSV) alongside unstructured text
3. **Extract and utilize metadata** from various file formats
4. **Create domain-specific tools** for specialized queries

5. **Implement progressive enhancements** from simple to advanced RAG
6. **Evaluate performance** with domain-specific metrics
7. **Build multi-modal reasoning** combining statistics, rules, and educational content
8. **Measure quality and performance KPIs** for quantitative agent comparison
9. **Analyze quality vs. performance trade-offs** for production deployment decisions

---

📋 Three Progressive Implementations

Implementation Progression

```
┌─────────────────────────────────────────────────────────────────┐
│  Level 1: Simple RAG                                            │
│  - Load CSV, HTML, TXT files                                    │
│  - Extract basic metadata                                       │
│  - Answer volleyball domain questions                           │
│  - Evaluate groundedness, relevance, completeness               │
└─────────────────────────────────────────────────────────────────┘
                                ↓
┌─────────────────────────────────────────────────────────────────┐
│  Level 2: Agentic RAG with smolagents                           │
│  + Reasoning and tool usage                                     │
│  + CSV analyzer tool for statistics                             │
│  + Rules comparison tool                                        │
│  + Multi-document reasoning                                     │
│  + Domain-specific evaluation agents                            │
│  + Use of smolagents for exploration                            │
└─────────────────────────────────────────────────────────────────┘
                                ↓
┌─────────────────────────────────────────────────────────────────┐
│  Level 3: Advanced Agentic RAG with LangChain                   │
│  + Hypothetical question generation                             │
│  + Hybrid search (BM25 + Vector)                                │
│  + Cross-encoder re-ranking                                     │
│  + LLM contextual compression                                   │
│  + Advanced CSV integration with pandas                         │
│  + Statistical analysis tools                                   │
│  + Cross-document reasoning                                     │
│  + Metadata-based filtering                                     │
│  + KPI-based evaluation (quality metrics + performance)         │
│  + Agent comparison framework (V1, V3, V5)                      │
│  + Production-ready deployment                                  │
└─────────────────────────────────────────────────────────────────┘
```

📘 Notebook 1: Simple RAG with VolleyBall Dataset

Filename: `RAG_with_Local_LLM_and_Embeddings_VolleyBall.ipynb`

Reference: Based on `RAG_with_Local_LLM_and_Embeddings.ipynb`

Objectives

- Implement foundational RAG pipeline with volleyball data
- Handle multiple file formats (CSV, HTML, TXT)
- Extract and utilize metadata from each format
- Answer domain-specific questions about volleyball
- Evaluate system performance on volleyball queries

Section-by-Section Implementation Guide

Section 1: Prerequisites and Installation

Instructions:

- Copy the installation section from the original notebook
- No changes needed - same dependencies

Code:

```
# Same as original notebook
pip install torch torchvision transformers sentence-transformers langchain chrc
```

Section 2: Import Required Libraries

Instructions:

- Add pandas import for CSV handling
- Add date/time parsing utilities

Additional Imports:

```
import pandas as pd
from datetime import datetime
import csv
```

Explanation:

- `pandas`: Essential for reading and analyzing CSV data
- `datetime`: For parsing and formatting dates from CSV

- `csv`: For low-level CSV operations if needed

Section 3: Load Local Models (LLM & Embeddings)

Instructions:

- Same as original notebook
- Load Qwen2.5 and all-MiniLM-L6-v2

Additional Notes:

- Consider using Qwen2.5-3B-Instruct for better reasoning
- Ensure 4-bit quantization for memory efficiency

Section 4: **NEW - CSV File Loader with Advanced Metadata Extraction**

🎯 **Challenge 1: Load CSV Files with Rich Metadata**

Instructions:

Create a specialized CSV loader that:

1. Reads the volleyball matches CSV file
2. Extracts comprehensive metadata (column names, data types, date ranges, team names)
3. Converts tabular rows into text chunks suitable for embedding
4. Adds rich metadata to each chunk for filtering and retrieval

Implementation Template:

```python
def extract_csv_metadata(file_path: str) -> Dict[str, Any]:
    """
    Extract comprehensive metadata from CSV file.

    Extracts:
    - File information (name, size, modification date)
    - Data schema (column names, data types, row count)
    - Data insights (date range, unique values, statistics)

    Args:
        file_path: Path to CSV file

    Returns:
        Dictionary containing metadata
    """
    from pathlib import Path

    metadata = {
        'source_type': 'csv',
        'file_path': file_path,
        'file_name': Path(file_path).name,
        'file_size_kb': Path(file_path).stat().st_size / 1024,
        'modified_date': datetime.fromtimestamp(
            Path(file_path).stat().st_mtime
        ).strftime('%Y-%m-%d %H:%M:%S'),
    }

    try:
        # Read CSV
        df = pd.read_csv(file_path)

        # Basic schema info
        metadata['row_count'] = len(df)
        metadata['column_count'] = len(df.columns)
        metadata['column_names'] = df.columns.tolist()
        metadata['data_types'] = df.dtypes.astype(str).to_dict()

        # Extract data insights
        # TODO: Identify date columns and extract date range
        # TODO: Identify categorical columns and extract unique values
        # TODO: Identify numeric columns and calculate basic statistics
        # TODO: Extract team names if present
        # TODO: Extract location information if present

        # Example: Date range extraction
        date_columns = [col for col in df.columns if 'date' in col.lower()]
        if date_columns:
            for col in date_columns:
                try:
                    dates = pd.to_datetime(df[col], errors='coerce')
                    metadata[f'{col}_min'] = dates.min().strftime('%Y-%m-%d')
                    metadata[f'{col}_max'] = dates.max().strftime('%Y-%m-%d')
```

```
                metadata[f'{col}_max'] = dates.max().strftime('%Y-%m-%d')
            except:
                pass

        # Example: Extract unique team names
        team_columns = [col for col in df.columns if 'team' in col.lower()]
        if team_columns:
            metadata['teams'] = []
            for col in team_columns:
                metadata['teams'].extend(df[col].dropna().unique().tolist())
            metadata['teams'] = list(set(metadata['teams']))[:20]  # Limit to 2

        # Statistical summary for numeric columns
        numeric_cols = df.select_dtypes(include=['number']).columns
        if len(numeric_cols) > 0:
            metadata['numeric_columns'] = numeric_cols.tolist()
            # Store basic stats (mean, min, max) for each numeric column
            stats_summary = df[numeric_cols].describe().to_dict()
            metadata['statistics_summary'] = stats_summary

    except Exception as e:
        print(f"Warning: Could not fully extract CSV metadata: {e}")
        metadata['error'] = str(e)

    return metadata


def csv_to_text_chunks(
    file_path: str,
    chunk_by: str = 'row',  # 'row' or 'group'
    group_size: int = 5
) -> List[str]:
    """
    Convert CSV data into text chunks suitable for embedding.

    Strategies:
    - 'row': Each row becomes a text description
    - 'group': Multiple rows grouped together

    Args:
        file_path: Path to CSV file
        chunk_by: Chunking strategy ('row' or 'group')
        group_size: Number of rows per group if using 'group' strategy

    Returns:
        List of text chunks
    """
    df = pd.read_csv(file_path)
    text_chunks = []

    if chunk_by == 'row':
        # Convert each row to natural language description
```

```
        for idx, row in df.iterrows():
            # Create a natural language representation
            text = f"Match Record {idx + 1}:\n"
            for col, value in row.items():
                if pd.notna(value):
                    text += f"- {col}: {value}\n"
            text_chunks.append(text)

    elif chunk_by == 'group':
        # Group multiple rows together
        for i in range(0, len(df), group_size):
            group = df.iloc[i:i+group_size]
            text = f"Match Records {i+1} to {i+len(group)}:\n\n"
            for idx, row in group.iterrows():
                text += f"Match {idx + 1}: "
                text += " | ".join([f"{col}={val}" for col, val in row.items()
                text += "\n"
            text_chunks.append(text)

    return text_chunks


def load_and_chunk_csv_files(
    file_paths: List[str],
    chunk_strategy: str = 'row'
) -> List[Document]:
    """
    Load CSV files, extract metadata, and create Document objects for embedding

    Args:
        file_paths: List of CSV file paths
        chunk_strategy: How to chunk CSV data ('row' or 'group')

    Returns:
        List of LangChain Document objects with content and metadata
    """
    from langchain_core.documents import Document

    all_documents = []

    for file_path in file_paths:
        print(f"Processing CSV: {file_path}")

        try:
            # Extract metadata
            metadata = extract_csv_metadata(file_path)

            # Convert to text chunks
            text_chunks = csv_to_text_chunks(file_path, chunk_by=chunk_strategy

            # Create Document objects
            for idx, chunk_text in enumerate(text_chunks):
```

```
                doc = Document(
                    page_content=chunk_text,
                    metadata={
                        **metadata,
                        'chunk_index': idx,
                        'total_chunks': len(text_chunks),
                        'chunk_type': f'csv_{chunk_strategy}'
                    }
                )
                all_documents.append(doc)

            print(f"  ✓ Created {len(text_chunks)} chunks from {metadata['row_

        except Exception as e:
            print(f"  ✗ Error processing {file_path}: {e}")

    return all_documents


# Usage:
csv_files = glob.glob("../UnstructuredDataVolleyBall/CSVFiles/*.csv")
csv_documents = load_and_chunk_csv_files(csv_files, chunk_strategy='row')

print(f"\nTotal CSV documents: {len(csv_documents)}")
print(f"\nSample CSV chunk:")
print(csv_documents[0].page_content[:300] if csv_documents else "No CSV documer
```

Key Concepts to Explain:

1. **CSV to Text Conversion:**
   - Tabular data must be converted to natural language for embedding
   - Each row can become a descriptive text chunk
   - Grouping strategies balance granularity vs. context
2. **Metadata Richness:**
   - Captures schema information (columns, types)
   - Extracts data insights (date ranges, unique values)
   - Enables filtering by team, date, location, etc.
3. **Chunking Strategies:**
   - **Row-level**: Fine-grained, good for specific match queries
   - **Group-level**: More context, good for comparative queries

---

Section 5: Load and Chunk HTML Files

🎯 Challenge 2: Extract Rules-Specific Metadata

Instructions:

HTML Proper Scraping:

1. Ensure proper parsing and demostrate by querying Chroma database

Enhance HTML loading to extract volleyball rules-specific metadata:

1. Rule source (organization name)
2. Rule category (indoor, beach, sand, grass)
3. Competition level (adult, junior, recreational)
4. Last updated date (if available)

Implementation Template:

```python
def extract_html_metadata_volleyball(file_path: str) -> Dict[str, Any]:
    """
    Extract volleyball rules-specific metadata from HTML files.

    Extracts:
    - Standard file metadata
    - Rule source/organization
    - Volleyball type (beach, indoor, sand, grass)
    - Competition level (adult, junior, recreational)
    - Rule sections/categories

    Args:
        file_path: Path to HTML file

    Returns:
        Dictionary containing metadata
    """
    from pathlib import Path
    from bs4 import BeautifulSoup

    metadata = {
        'source_type': 'html',
        'file_path': file_path,
        'file_name': Path(file_path).name,
        'file_size_kb': Path(file_path).stat().st_size / 1024,
        'modified_date': datetime.fromtimestamp(
            Path(file_path).stat().st_mtime
        ).strftime('%Y-%m-%d'),
    }

    try:
        # Parse HTML
        with open(file_path, 'r', encoding='utf-8') as f:
            soup = BeautifulSoup(f.read(), 'html.parser')

        # Extract title
        title = soup.find('title')
        if title:
            metadata['html_title'] = title.get_text().strip()
        else:
            metadata['html_title'] = Path(file_path).stem

        # TODO: Identify volleyball type from filename/content
        filename_lower = Path(file_path).name.lower()
        if 'beach' in filename_lower:
            metadata['volleyball_type'] = 'beach'
        elif 'sand' in filename_lower:
            metadata['volleyball_type'] = 'sand'
        elif 'grass' in filename_lower:
            metadata['volleyball_type'] = 'grass'
        elif 'indoor' in filename_lower:
            metadata['volleyball_type'] = 'indoor'
```

```
            metadata['volleyball_type'] = 'indoor'
        else:
            metadata['volleyball_type'] = 'general'

        # TODO: Identify competition level
        if 'adult' in filename_lower:
            metadata['competition_level'] = 'adult'
        elif 'junior' in filename_lower or 'youth' in filename_lower:
            metadata['competition_level'] = 'junior'
        else:
            metadata['competition_level'] = 'general'

        # TODO: Extract organization name
        # Look for common patterns in HTML content
        text_content = soup.get_text().lower()
        if 'canon city' in text_content:
            metadata['organization'] = 'Canon City Area Metropolitan Recreatior
        elif 'volley life' in text_content:
            metadata['organization'] = 'Volley Life'
        else:
            metadata['organization'] = 'Unknown'

        # Extract headings for rule categories
        headings = soup.find_all(['h1', 'h2', 'h3'])
        if headings:
            metadata['rule_sections'] = [h.get_text().strip() for h in headings

    except Exception as e:
        print(f"Warning: Could not fully extract HTML metadata: {e}")
        metadata['html_title'] = Path(file_path).stem
        metadata['volleyball_type'] = 'unknown'

    return metadata


def load_and_chunk_html_volleyball(
    file_paths: List[str],
    chunk_size: int = 512,
    chunk_overlap: int = 50
) -> List[Document]:
    """
    Load volleyball rules HTML files with enhanced metadata.

    Args:
        file_paths: List of HTML file paths
        chunk_size: Maximum characters per chunk
        chunk_overlap: Overlap between chunks

    Returns:
        List of Document objects
    """
    from langchain_community.document_loaders import UnstructuredHTMLLoader
```

```python
    from langchain_text_splitters import RecursiveCharacterTextSplitter

    text_splitter = RecursiveCharacterTextSplitter(
        chunk_size=chunk_size,
        chunk_overlap=chunk_overlap,
        length_function=len,
        separators=["\n\n", "\n", ". ", " ", ""]
    )

    all_documents = []

    for file_path in file_paths:
        print(f"Processing HTML: {file_path}")

        try:
            # Extract metadata
            metadata = extract_html_metadata_volleyball(file_path)

            # Load HTML
            loader = UnstructuredHTMLLoader(file_path)
            docs = loader.load()

            # Add metadata to documents
            for doc in docs:
                doc.metadata.update(metadata)

            # Chunk
            chunks = text_splitter.split_documents(docs)

            # Add chunk indices
            for idx, chunk in enumerate(chunks):
                chunk.metadata['chunk_index'] = idx
                chunk.metadata['total_chunks'] = len(chunks)

            all_documents.extend(chunks)
            print(f"  ✓ Created {len(chunks)} chunks")

        except Exception as e:
            print(f"  ✗ Error processing {file_path}: {e}")

    return all_documents


# Usage:
html_files = glob.glob("../UnstructuredDataVolleyBall/HTMLFiles/*.html")
html_documents = load_and_chunk_html_volleyball(html_files)

print(f"\nTotal HTML documents: {len(html_documents)}")
```

Key Concepts:

- Domain-specific metadata extraction
- Using filename patterns for categorization
- Text parsing for organization identification
- Structured metadata for filtering

Section 6: Load and Chunk Text Files

Instructions:

Load text files (lesson plans and rules) with appropriate metadata:

```python
def extract_text_metadata_volleyball(file_path: str) -> Dict[str, Any]:
    """
    Extract metadata from volleyball text files.

    Args:
        file_path: Path to text file

    Returns:
        Dictionary containing metadata
    """
    from pathlib import Path

    metadata = {
        'source_type': 'text',
        'file_path': file_path,
        'file_name': Path(file_path).name,
        'file_size_kb': Path(file_path).stat().st_size / 1024,
        'modified_date': datetime.fromtimestamp(
            Path(file_path).stat().st_mtime
        ).strftime('%Y-%m-%d'),
    }

    filename_lower = Path(file_path).name.lower()

    # Identify content type
    if 'lesson' in filename_lower or 'plan' in filename_lower:
        metadata['content_type'] = 'lesson_plan'
        metadata['document_category'] = 'educational'
    elif 'rule' in filename_lower:
        metadata['content_type'] = 'rules'
        metadata['document_category'] = 'regulatory'
    else:
        metadata['content_type'] = 'general'
        metadata['document_category'] = 'general'

    # Identify volleyball type
    if 'grass' in filename_lower:
        metadata['volleyball_type'] = 'grass'
    elif 'beach' in filename_lower:
        metadata['volleyball_type'] = 'beach'
    elif 'sand' in filename_lower:
        metadata['volleyball_type'] = 'sand'
    else:
        metadata['volleyball_type'] = 'general'

    # Identify level
    if 'junior' in filename_lower or 'high' in filename_lower:
        metadata['competition_level'] = 'junior'
    else:
        metadata['competition_level'] = 'general'

    return metadata
```

```
    return metadata


# Load text files
text_files = glob.glob("../UnstructuredDataVolleyBall/TextFiles/*.txt")
text_documents = []

for file_path in text_files:
    try:
        metadata = extract_text_metadata_volleyball(file_path)
        loader = TextLoader(file_path, encoding='utf-8')
        docs = loader.load()

        for doc in docs:
            doc.metadata.update(metadata)

        chunks = text_splitter.split_documents(docs)

        for idx, chunk in enumerate(chunks):
            chunk.metadata['chunk_index'] = idx
            chunk.metadata['total_chunks'] = len(chunks)

        text_documents.extend(chunks)
        print(f"✓ Loaded: {Path(file_path).name} ({len(chunks)} chunks)")

    except Exception as e:
        print(f"✗ Error loading {file_path}: {e}")

print(f"\nTotal text documents: {len(text_documents)}")
```

Section 7: Combine All Documents and Create Vector Store

Instructions:

```python
# Combine all documents
all_documents = csv_documents + html_documents + text_documents

print(f"Total documents from all sources:")
print(f"  - CSV documents: {len(csv_documents)}")
print(f"  - HTML documents: {len(html_documents)}")
print(f"  - Text documents: {len(text_documents)}")
print(f"  - TOTAL: {len(all_documents)}")

# Create vector store
print("\nCreating vector store...")
vectorstore = Chroma.from_documents(
    documents=all_documents,
    embedding=embedding_model,
    collection_name='VolleyBall_Documentation_RAG',
    persist_directory='./chroma_db_volleyball'
)

print("✓ Vector store created successfully!")
print(f"Documents indexed: {vectorstore._collection.count()}")
```

Section 8: Create RAG Function

🎯 Challenge 3: Domain-Specific System Prompt

Instructions:

Create a system prompt tailored for volleyball domain:

```
volleyball_system_message = """You are a knowledgeable volleyball assistant wit

Your knowledge base includes:
- Official volleyball rules for different formats (beach, sand, grass, indoor)
- Match statistics and game data
- Lesson plans and coaching materials
- Regulations from various organizations

User input will have the context required to answer questions, beginning with #
User questions will begin with ###Question.

Instructions:
- Answer questions ONLY using the information provided in the context
- Do not mention or reference the context in your final answer
- Provide clear, accurate answers about volleyball rules, statistics, and coach
- When discussing rules, specify the volleyball format (beach/sand/grass/indoor
- When citing statistics, mention the relevant teams, dates, or matches if avai
- If the answer is not found in the context, respond with "I don't know based c
- Be helpful and educational in your responses
- Use proper volleyball terminology

Your goal is to help users understand volleyball better, whether they are playe

print("✓ Volleyball-specific system message configured")
```

Section 9: Test with Volleyball-Specific Queries

🎯 Challenge 4: Diverse Query Types

Instructions:

Test the RAG system with various volleyball-specific queries:

```python
# Define diverse volleyball queries
volleyball_queries = [
    # Rules queries
    "What are the serving rules in beach volleyball?",
    "How many players are on a volleyball team?",
    "What is the scoring system in volleyball?",

    # Statistics queries (for CSV data)
    "What matches have been played and what were the results?",
    "Which teams have played the most games?",

    # Educational queries
    "What are some volleyball drills for junior high students?",
    "How should beginners learn volleyball fundamentals?",

    # Comparative queries
    "What are the differences between beach and grass volleyball rules?",

    # Specific rule queries
    "What are the court dimensions for volleyball?",
    "What happens if the ball touches the net during service?"
]

# Test RAG system
print("=" * 80)
print("TESTING VOLLEYBALL RAG SYSTEM")
print("=" * 80)

for i, query in enumerate(volleyball_queries[:5], 1):  # Test first 5
    print(f"\nQuery {i}: {query}")
    print("-" * 80)

    answer = RAG(query)
    print(f"Answer: {answer}")
    print("=" * 80)
```

Section 10: **NEW - Evaluation with Volleyball Domain Metrics**

🎯 Challenge 5: Domain-Specific Evaluation

Instructions:

In addition to the standard three metrics (groundedness, context relevance, answer relevance), add:

1. **Terminology Accuracy**: Does the answer use correct volleyball terminology?
2. **Rule Specificity**: For rule-related questions, does it specify the volleyball format?
3. **Data Source Attribution**: For statistics, are sources (teams, dates) mentioned?

Implementation Template:

```python
def evaluate_volleyball_terminology(question: str, answer: str) -> str:
    """
    Evaluate if the answer uses correct volleyball terminology.

    Checks for:
    - Proper volleyball terms (serve, spike, dig, set, block, etc.)
    - Correct format terminology (beach, indoor, sand, grass)
    - Proper scoring terms (rally scoring, side-out, etc.)

    Args:
        question: The question asked
        answer: The generated answer

    Returns:
        Evaluation result
    """
    evaluation_prompt = f"""You are an expert volleyball official evaluating te

**Question**: {question}

**Answer**: {answer}

**Task**: Evaluate if the answer uses correct volleyball terminology.

**Criteria**:
1. Are volleyball-specific terms used correctly? (serve, spike, dig, set, block
2. Is the correct format specified when relevant? (beach, indoor, sand, grass)
3. Are scoring terms accurate? (rally scoring, side-out, point, match, set)
4. Are position terms correct if mentioned? (setter, libero, outside hitter, mi
5. Are violation terms accurate if mentioned? (foot fault, net violation, doubl

**Evaluation** (1-5 scale):
1 - Incorrect or vague terminology
2 - Some correct terms but many errors or vague descriptions
3 - Mostly correct terminology with minor issues
4 - Correct terminology with good specificity
5 - Excellent use of precise volleyball terminology

Provide:
- Analysis of terminology usage (2-3 sentences)
- List of correct terms used
- List of any terminology errors or vague descriptions
- Final score (1-5)
"""

    response = llm_pipeline(evaluation_prompt, max_new_tokens=400, temperature=
    return response[0]['generated_text'][len(evaluation_prompt):].strip()


def evaluate_rule_specificity(question: str, answer: str, context: str) -> str:
    """
    For rule-related questions, evaluate if the answer specifies the volleyball
```

```
    For rule related questions, evaluate if the answer specifies the volleyball

    Args:
        question: The question asked
        answer: The generated answer
        context: The retrieved context

    Returns:
        Evaluation result
    """
    evaluation_prompt = f"""You are an expert evaluating volleyball rule specif

**Question**: {question}

**Context Available**:
{context[:1000]}... [truncated]

**Answer**: {answer}

**Task**: Evaluate if the answer properly specifies the volleyball format when

**Considerations**:
- Rules often differ between formats (beach, indoor, sand, grass)
- If context contains format-specific information, answer should reflect it
- General rules should be stated as "generally" or "in most formats"
- Format-specific rules should clearly state the format

**Evaluation** (1-5 scale):
1 - Ignores format differences, provides incorrect generalization
2 - Vague about format, doesn't distinguish when it should
3 - Mentions format but could be more specific
4 - Good format specificity where relevant
5 - Excellent format specificity with clear distinctions

Provide:
- Analysis (2 sentences)
- Format specificity assessment
- Final score (1-5)
"""

    response = llm_pipeline(evaluation_prompt, max_new_tokens=400, temperature=
    return response[0]['generated_text'][len(evaluation_prompt):].strip()


# Comprehensive evaluation function
def evaluate_volleyball_rag_response(question: str, answer: str, context: str)
    """
    Comprehensive evaluation for volleyball RAG responses.

    Evaluates:
    1. Groundedness (standard)
    2. Context Relevance (standard)
```

```
    3. Answer Relevance (standard)
    4. Terminology Accuracy (volleyball-specific)
    5. Rule Specificity (volleyball-specific, for rule questions)

    Args:
        question: The question
        answer: The generated answer
        context: The retrieved context

    Returns:
        Dictionary with all evaluation results
    """
    evaluations = {}

    print("Evaluating: Groundedness...")
    evaluations['groundedness'] = evaluate_groundedness(question, context, answ

    print("Evaluating: Context Relevance...")
    evaluations['context_relevance'] = evaluate_context_relevance(question, cor

    print("Evaluating: Answer Relevance...")
    evaluations['answer_relevance'] = evaluate_answer_relevance(question, answe

    print("Evaluating: Terminology Accuracy...")
    evaluations['terminology_accuracy'] = evaluate_volleyball_terminology(quest

    # Only evaluate rule specificity for rule-related questions
    if any(word in question.lower() for word in ['rule', 'regulation', 'legal',
        print("Evaluating: Rule Specificity...")
        evaluations['rule_specificity'] = evaluate_rule_specificity(question, a

    return evaluations


# Example usage
sample_query = volleyball_queries[0]  # "What are the serving rules in beach vo
sample_answer = RAG(sample_query)
sample_context = "\n\n".join([doc.page_content for doc in retriever.invoke(samp

print(f"\nQuery: {sample_query}")
print(f"\nAnswer: {sample_answer}")
print("\n" + "=" * 80)
print("COMPREHENSIVE EVALUATION")
print("=" * 80)

eval_results = evaluate_volleyball_rag_response(sample_query, sample_answer, sa

for metric, result in eval_results.items():
    print(f"\n--- {metric.upper().replace('_', ' ')} ---")
    print(result)
    print("-" * 80)
```

Key Concepts:

- Domain-specific evaluation goes beyond generic metrics
- Terminology accuracy ensures professional quality
- Format specificity prevents rule confusion
- Multiple evaluation dimensions provide comprehensive quality assessment

---

Section 11: **NEW - Metadata-Based Filtering**

🎯 **Challenge 6: Filtered Retrieval**

Instructions:

Demonstrate how to filter retrieval by metadata:

```python
def filtered_rag_query(
    question: str,
    filters: Dict[str, Any] = None
) -> str:
    """
    RAG query with metadata filtering.

    Args:
        question: User's question
        filters: Metadata filters (e.g., {'volleyball_type': 'beach'})

    Returns:
        Generated answer
    """
    # Retrieve with filters
    if filters:
        # Create filtered retriever
        filtered_retriever = vectorstore.as_retriever(
            search_type='similarity',
            search_kwargs={'k': 5, 'filter': filters}
        )
        relevant_docs = filtered_retriever.invoke(question)
    else:
        relevant_docs = retriever.invoke(question)

    # Generate answer
    context = "\n\n".join([doc.page_content for doc in relevant_docs])
    prompt = f"{volleyball_system_message}\n\n###Context\n{context}\n\n###Quest

    response = llm_pipeline(prompt, max_new_tokens=512, temperature=0.1)
    return response[0]['generated_text'][len(prompt):].strip()


# Test filtered queries
print("\n" + "=" * 80)
print("TESTING FILTERED RETRIEVAL")
print("=" * 80)

# Query 1: Only beach volleyball rules
print("\nQuery: What are the rules? (Filter: beach volleyball only)")
answer1 = filtered_rag_query(
    "What are the serving rules?",
    filters={'volleyball_type': 'beach'}
)
print(f"Answer: {answer1}")

# Query 2: Only CSV data (statistics)
print("\n" + "-" * 80)
print("\nQuery: What matches have been played? (Filter: CSV data only)")
answer2 = filtered_rag_query(
    "What matches have been played?",
    filters={'source_type': 'csv'}
```

```
    filters={'source_type': 'csv'}
)
print(f"Answer: {answer2}")

# Query 3: Only educational content
print("\n" + "-" * 80)
print("\nQuery: How to teach volleyball? (Filter: educational content only)")
answer3 = filtered_rag_query(
    "What are good drills for beginners?",
    filters={'document_category': 'educational'}
)
print(f"Answer: {answer3}")

print("\n" + "=" * 80)
```

Key Concepts:

- Metadata enables targeted retrieval
- Filters improve precision for specific query types
- Users can specify source types or domains
- Reduces noise from irrelevant documents

Summary of Notebook 1 Challenges

Completed Challenges:

1. ✅ **CSV Loading with Rich Metadata** - Extract schema, statistics, and data insights
2. ✅ **Rules-Specific HTML Metadata** - Extract volleyball format, level, organization
3. ✅ **Domain-Specific System Prompt** - Volleyball-tailored instructions
4. ✅ **Diverse Query Types** - Rules, statistics, educational, comparative
5. ✅ **Volleyball Domain Evaluation** - Terminology accuracy, rule specificity
6. ✅ **Metadata-Based Filtering** - Targeted retrieval by source/type/format

Learning Outcomes:

- Handling tabular data (CSV) in RAG systems
- Domain-specific metadata extraction
- Creating specialized evaluation metrics
- Implementing filtered retrieval

📗 Notebook 2: Agentic RAG with VolleyBall Dataset (smolagents)

Filename: `RAG_with_Agentic_RAG_and_Embeddings_VolleyBall.ipynb`

Reference: Based on `RAG_with_Agentic_RAG_and_Embeddings.ipynb`

Objectives

- Add reasoning and tool usage with smolagents
- Create volleyball-specific tools
- Implement multi-agent architecture
- Handle complex multi-step queries
- Evaluate agent reasoning quality

Additional Challenges Beyond Notebook 1

Section 1-7: Base Setup

Instructions:

- Replicate Sections 1-7 from Notebook 1 (installation, imports, model loading, document loading)
- Same CSV/HTML/TXT loading with metadata

Section 8: **NEW - Create CSV Analyzer Tool**

🎯 **Challenge 7: Statistical Analysis Tool**

Instructions:

Create a specialized tool that can analyze CSV data and answer statistical questions:

```python
class VolleyBallCSVAnalyzer(Tool):
    """
    Tool for analyzing volleyball match statistics from CSV data.

    This tool can:
    - Calculate team statistics (wins, losses, scores)
    - Find matches by team, date, or criteria
    - Compare team performance
    - Extract trends and patterns
    - Answer statistical questions

    The tool uses pandas for data analysis and natural language
    generation to present findings.
    """

    name = "volleyball_statistics_analyzer"

    description = """Analyzes volleyball match statistics and game data.
    Use this tool to answer questions about:
    - Match results and scores
    - Team performance and statistics
    - Win/loss records
    - Score comparisons
    - Date-based queries
    - Statistical trends

    Input should be a clear question about match data or team statistics.
    Returns analysis results in natural language."""

    inputs = {
        "query": {
            "type": "string",
            "description": "Question about volleyball statistics or match data"
        }
    }
    output_type = "string"

    def __init__(self, csv_file_path: str, **kwargs):
        """
        Initialize the CSV analyzer tool.

        Args:
            csv_file_path: Path to the volleyball matches CSV file
        """
        super().__init__(**kwargs)
        self.csv_path = csv_file_path
        self.df = pd.read_csv(csv_file_path)

        # Analyze CSV structure
        self.columns = self.df.columns.tolist()
        print(f"CSV Analyzer initialized with {len(self.df)} rows")
        print(f"Columns: {', '.join(self.columns)}")
```

```
        print(f"Columns: {', '.join(self.columns)}")

    def forward(self, query: str) -> str:
        """
        Analyze CSV data based on the query.

        This method:
        1. Parses the query to understand what's being asked
        2. Performs relevant pandas operations
        3. Formats results as natural language

        Args:
            query: Natural language question about the data

        Returns:
            Analysis results as formatted text
        """
        try:
            query_lower = query.lower()

            result = f"Analysis Results for: {query}\n\n"

            # TODO: Implement query understanding and data analysis
            # Examples:

            # 1. Team-based queries
            if 'team' in query_lower:
                # Extract team mentions
                # Show team statistics
                result += "Team Statistics:\n"
                # Add team analysis...

            # 2. Date-based queries
            elif 'when' in query_lower or 'date' in query_lower:
                # Filter by date
                # Show chronological results
                result += "Date-based Results:\n"
                # Add date analysis...

            # 3. Score/result queries
            elif 'score' in query_lower or 'result' in query_lower:
                # Show scores and outcomes
                result += "Match Results:\n"
                # Add score analysis...

            # 4. Comparative queries
            elif 'vs' in query_lower or 'versus' in query_lower or 'compare' ir
                # Compare teams or matches
                result += "Comparison Results:\n"
                # Add comparison...

            # 5. General statistics
```

```
            else:
                # Provide overview
                result += f"Dataset Overview:\n"
                result += f"- Total matches: {len(self.df)}\n"
                result += f"- Columns: {', '.join(self.columns)}\n"

                # Add summary statistics
                numeric_cols = self.df.select_dtypes(include=['number']).columr
                if len(numeric_cols) > 0:
                    result += f"\nNumeric Column Statistics:\n"
                    for col in numeric_cols:
                        result += f"- {col}: "
                        result += f"Mean={self.df[col].mean():.2f}, "
                        result += f"Min={self.df[col].min():.2f}, "
                        result += f"Max={self.df[col].max():.2f}\n"

            # Add relevant data samples
            result += "\nRelevant Data Sample:\n"
            result += self.df.head(3).to_string()

            return result

        except Exception as e:
            return f"Error analyzing data: {str(e)}\nPlease rephrase your quest


# Create the tool
csv_analyzer = VolleyBallCSVAnalyzer(
    csv_file_path="../UnstructuredDataVolleyBall/CSVFiles/vb_matches.csv"
)

print("✓ CSV Analyzer tool created!")
```

Key Concepts:

- Tools encapsulate specific capabilities
- Pandas integration for data analysis
- Natural language output from structured data
- Error handling for robust tool operation

Section 9: **NEW - Create Rules Comparison Tool**

🎯 **Challenge 8: Multi-Document Reasoning Tool**

Instructions:

Create a tool that can compare rules across different volleyball formats:

```python
class VolleyBallRulesComparison(Tool):
    """
    Tool for comparing volleyball rules across different formats.

    This tool can:
    - Compare rules between beach, sand, grass, and indoor volleyball
    - Identify similarities and differences
    - Highlight format-specific regulations
    - Answer questions about rule variations

    The tool retrieves relevant rule documents and performs
    comparative analysis.
    """

    name = "volleyball_rules_comparison"

    description = """Compares volleyball rules across different formats.
    Use this tool to answer questions about:
    - Differences between beach and indoor rules
    - Format-specific regulations (court size, team size, scoring)
    - Common rules across all formats
    - When specific rules apply

    Input should clearly state what rules or formats to compare.
    Returns formatted comparison."""

    inputs = {
        "query": {
            "type": "string",
            "description": "Question about rule differences or comparisons"
        }
    }
    output_type = "string"

    def __init__(self, vectorstore, **kwargs):
        """
        Initialize the rules comparison tool.

        Args:
            vectorstore: ChromaDB vectorstore with volleyball documents
        """
        super().__init__(**kwargs)
        self.vectorstore = vectorstore

    def forward(self, query: str) -> str:
        """
        Compare rules based on the query.

        This method:
        1. Identifies volleyball formats mentioned in query
        2. Retrieves rules for each format
        3. Compares and contrasts the rules
```

```
    3. Compares and contrasts the rules
    4. Formats results for easy understanding

    Args:
        query: Question about rule comparisons

    Returns:
        Comparison results
    """
    try:
        result = f"Rules Comparison for: {query}\n\n"

        # Identify formats mentioned
        query_lower = query.lower()
        formats = []
        if 'beach' in query_lower:
            formats.append('beach')
        if 'sand' in query_lower:
            formats.append('sand')
        if 'grass' in query_lower:
            formats.append('grass')
        if 'indoor' in query_lower:
            formats.append('indoor')

        if not formats:
            formats = ['beach', 'sand', 'grass', 'indoor']  # Compare all

        # Retrieve rules for each format
        rules_by_format = {}
        for format_type in formats:
            # Query vectorstore with format filter
            docs = self.vectorstore.similarity_search(
                query,
                k=3,
                filter={'volleyball_type': format_type}
            )

            if docs:
                rules_by_format[format_type] = "\n".join([doc.page_content
            else:
                rules_by_format[format_type] = "No specific rules found for

        # Format comparison results
        if len(rules_by_format) == 0:
            result += "No rules found for comparison.\n"
        else:
            result += f"Comparing {len(rules_by_format)} format(s): {', '.j

            for format_type, rules_text in rules_by_format.items():
                result += f"=== {format_type.upper()} VOLLEYBALL ===\n"
                result += rules_text[:500] + "...\n\n"
```

```
            result += "\nNote: Use these rule excerpts to identify similarities

            return result

        except Exception as e:
            return f"Error comparing rules: {str(e)}"


# Create the tool
rules_comparison_tool = VolleyBallRulesComparison(vectorstore=vectorstore)

print("✓ Rules Comparison tool created!")
```

Section 10: **NEW - Create Main Volleyball Agent**

🎯 **Challenge 9: Multi-Tool Agent**

Instructions:

Create an agent that can use multiple tools (retrieval + CSV analyzer + rules comparison):

```python
# Create retrieval tool (standard)
class VolleyBallKnowledgeSearch(Tool):
    """Standard retrieval tool for volleyball knowledge base."""

    name = "volleyball_knowledge_search"
    description = """Search the volleyball knowledge base for information about
    techniques, coaching, and general volleyball knowledge.
    Use this for general questions that don't require statistical analysis or r

    inputs = {"query": {"type": "string", "description": "Search query"}}
    output_type = "string"

    def __init__(self, vectorstore, **kwargs):
        super().__init__(**kwargs)
        self.vectorstore = vectorstore

    def forward(self, query: str) -> str:
        docs = self.vectorstore.similarity_search(query, k=5)

        result = f"Found {len(docs)} relevant documents:\n\n"
        for i, doc in enumerate(docs, 1):
            result += f"Document {i}:\n"
            result += f"Type: {doc.metadata.get('source_type', 'unknown')}\n"
            result += f"Content: {doc.page_content[:300]}...\n\n"

        return result


# Create all tools
knowledge_tool = VolleyBallKnowledgeSearch(vectorstore=vectorstore)
csv_analyzer_tool = VolleyBallCSVAnalyzer(
    csv_file_path="../UnstructuredDataVolleyBall/CSVFiles/vb_matches.csv"
)
rules_comparison_tool = VolleyBallRulesComparison(vectorstore=vectorstore)

# Create main agent with all tools
volleyball_agent = CodeAgent(
    tools=[knowledge_tool, csv_analyzer_tool, rules_comparison_tool],
    model=agent_model,
    max_steps=10,
    verbosity_level=2  # Show reasoning process
)

print("✓ Volleyball Agent created with 3 tools:")
print("  1. volleyball_knowledge_search")
print("  2. volleyball_statistics_analyzer")
print("  3. volleyball_rules_comparison")
```

Key Concepts:

- Multi-tool agents can handle diverse query types
- Agent decides which tool to use based on query
- Tools are composed to handle complex requests
- Verbose mode shows tool selection reasoning

Section 11: **NEW - Test Multi-Tool Agent**

🎯 **Challenge 10: Complex Query Handling**

Instructions:

Test the agent with queries that require different tools:

```python
# Complex test queries
complex_volleyball_queries = [
    # Requires: CSV Analyzer
    {
        'query': "Which team has won the most matches and what is their win rat
        'expected_tool': 'volleyball_statistics_analyzer',
        'complexity': 'statistical'
    },

    # Requires: Rules Comparison
    {
        'query': "What are the main differences between beach and grass volleyb
        'expected_tool': 'volleyball_rules_comparison',
        'complexity': 'comparative'
    },

    # Requires: Knowledge Search
    {
        'query': "What are some good drills for teaching volleyball to beginner
        'expected_tool': 'volleyball_knowledge_search',
        'complexity': 'educational'
    },

    # Requires: Multiple Tools
    {
        'query': "Based on match data, what rules should junior players focus c
        'expected_tool': 'multiple',
        'complexity': 'multi-step'
    },
]

print("=" * 100)
print("TESTING MULTI-TOOL VOLLEYBALL AGENT")
print("=" * 100)

for i, test_case in enumerate(complex_volleyball_queries, 1):
    print(f"\n{'='*100}")
    print(f"TEST {i}: {test_case['complexity'].upper()} QUERY")
    print(f"{'='*100}")
    print(f"Query: {test_case['query']}")
    print(f"Expected Tool: {test_case['expected_tool']}")
    print(f"\n{'→'*50}")
    print("AGENT REASONING PROCESS:")
    print(f"{'→'*50}\n")

    try:
        answer = volleyball_agent.run(test_case['query'])

        print(f"\n{'='*100}")
        print(f"FINAL ANSWER:")
        print(f"{'='*100}")
        print(answer)
```

```
        print(answer)
        print(f"\n{'='*100}\n")

    except Exception as e:
        print(f"Error: {e}\n")
```

Expected Behavior:

For each query, observe in verbose mode:

1. **Task Interpretation**: How the agent understands the query
2. **Tool Selection**: Which tool(s) the agent chooses and why
3. **Intermediate Results**: What data the tool returns
4. **Reasoning Steps**: How the agent processes the information
5. **Answer Synthesis**: How the final answer is composed

Section 12: **NEW - Create Evaluation Agents for Tool Usage**

🎯 Challenge 11: Agent Performance Evaluation

Instructions:

Create specialized evaluation agents to assess:

1. Tool selection appropriateness
2. Multi-step reasoning quality
3. Answer synthesis from multiple sources

```python
class ToolUsageEvaluator(Tool):
    """
    Evaluates whether the agent selected and used the appropriate tools.

    Assesses:
    - Was the right tool chosen for the query?
    - Were multiple tools used when needed?
    - Did the tool usage lead to better answers?
    """

    name = "tool_usage_evaluator"
    description = "Evaluates agent's tool selection and usage quality"

    inputs = {
        "query": {"type": "string", "description": "Original query"},
        "tools_used": {"type": "string", "description": "List of tools used"},
        "answer": {"type": "string", "description": "Final answer generated"}
    }
    output_type = "string"

    def __init__(self, llm_pipeline, **kwargs):
        super().__init__(**kwargs)
        self.llm_pipeline = llm_pipeline

    def forward(self, query: str, tools_used: str, answer: str) -> str:
        """Evaluate tool usage quality."""

        evaluation_prompt = f"""You are an expert evaluating AI agent tool usag

**Query**: {query}

**Tools Used**: {tools_used}

**Final Answer**: {answer}

**Available Tools**:
1. volleyball_knowledge_search - General knowledge retrieval
2. volleyball_statistics_analyzer - CSV data analysis
3. volleyball_rules_comparison - Multi-format rule comparison

**Evaluation Criteria** (1-5 scale):
1 - Wrong tool selection, inappropriate for query
2 - Suboptimal tool choice, could be better
3 - Adequate tool selection, works but not ideal
4 - Good tool selection, appropriate for query
5 - Excellent tool selection and usage, optimal approach

**Consider**:
- Statistical questions should use CSV analyzer
- Rule comparison questions should use rules comparison tool
- General questions should use knowledge search
- Complex questions may require multiple tools
```

```
  Complex questions may require multiple tools

Provide:
- Analysis of tool selection appropriateness (2-3 sentences)
- Whether multiple tools should have been used
- Suggestions for improvement
- Final score (1-5)
"""

        response = self.llm_pipeline(evaluation_prompt, max_new_tokens=400, tem
        return response[0]['generated_text'][len(evaluation_prompt):].strip()


class MultiStepReasoningEvaluator(Tool):
    """
    Evaluates the quality of multi-step reasoning in agentic RAG.

    Assesses:
    - Logical flow of reasoning steps
    - Effectiveness of information gathering
    - Quality of synthesis from multiple sources
    """

    name = "reasoning_quality_evaluator"
    description = "Evaluates multi-step reasoning quality"

    inputs = {
        "query": {"type": "string", "description": "Original query"},
        "reasoning_trace": {"type": "string", "description": "Agent's reasoning
        "answer": {"type": "string", "description": "Final answer"}
    }
    output_type = "string"

    def __init__(self, llm_pipeline, **kwargs):
        super().__init__(**kwargs)
        self.llm_pipeline = llm_pipeline

    def forward(self, query: str, reasoning_trace: str, answer: str) -> str:
        """Evaluate multi-step reasoning quality."""

        evaluation_prompt = f"""You are an expert evaluating AI reasoning quali

**Query**: {query}

**Reasoning Trace**: {reasoning_trace}

**Final Answer**: {answer}

**Evaluation Criteria** (1-5 scale):
1 - Poor reasoning: illogical, jumps to conclusions, missing steps
2 - Weak reasoning: some logic but significant gaps or errors
3 - Adequate reasoning: logical but could be more thorough
```

```
4 - Good reasoning: clear logical flow, well-structured
5 - Excellent reasoning: exemplary step-by-step logic, comprehensive

**Assess**:
- Is the reasoning logical and well-structured?
- Are intermediate steps clearly explained?
- Does the agent effectively gather information?
- Is the synthesis from multiple sources coherent?
- Are there any logical fallacies or gaps?

Provide:
- Analysis of reasoning quality (2-3 sentences)
- Strengths of the reasoning approach
- Areas for improvement
- Final score (1-5)
"""

        response = self.llm_pipeline(evaluation_prompt, max_new_tokens=400, tem
        return response[0]['generated_text'][len(evaluation_prompt):].strip()


# Create evaluation agents
tool_usage_evaluator = ToolUsageEvaluator(llm_pipeline=llm_pipeline)
reasoning_evaluator = MultiStepReasoningEvaluator(llm_pipeline=llm_pipeline)

evaluation_agent_tools = CodeAgent(
    tools=[tool_usage_evaluator, reasoning_evaluator],
    model=agent_model,
    max_steps=4,
    verbosity_level=2
)

print("✓ Evaluation agents created for tool usage and reasoning quality!")
```

Section 13: **NEW - Comprehensive Agent Evaluation**

Instructions:

Evaluate the volleyball agent's performance across multiple dimensions:

```python
def comprehensive_volleyball_agent_evaluation(
    query: str,
    answer: str,
    tools_used: List[str],
    reasoning_trace: str,
    context: str
) -> dict:
    """
    Comprehensive evaluation of volleyball agent performance.

    Evaluates:
    1. Groundedness (standard)
    2. Context Relevance (standard)
    3. Answer Relevance (standard)
    4. Terminology Accuracy (volleyball-specific)
    5. Tool Usage Quality (agent-specific)
    6. Reasoning Quality (agent-specific)

    Args:
        query: The question
        answer: The generated answer
        tools_used: List of tools the agent used
        reasoning_trace: Agent's reasoning steps
        context: Retrieved context

    Returns:
        Dictionary with all evaluation results
    """
    evaluations = {}

    # Standard evaluations
    print("Evaluating: Groundedness...")
    evaluations['groundedness'] = evaluate_groundedness(query, context, answer)

    print("Evaluating: Context Relevance...")
    evaluations['context_relevance'] = evaluate_context_relevance(query, contex

    print("Evaluating: Answer Relevance...")
    evaluations['answer_relevance'] = evaluate_answer_relevance(query, answer)

    # Volleyball-specific
    print("Evaluating: Terminology Accuracy...")
    evaluations['terminology'] = evaluate_volleyball_terminology(query, answer)

    # Agent-specific
    print("Evaluating: Tool Usage...")
    evaluations['tool_usage'] = tool_usage_evaluator.forward(
        query=query,
        tools_used=", ".join(tools_used),
        answer=answer
    )
```

```
    print("Evaluating: Reasoning Quality...")
    evaluations['reasoning'] = reasoning_evaluator.forward(
        query=query,
        reasoning_trace=reasoning_trace,
        answer=answer
    )

    return evaluations


# Example evaluation
print("\n" + "=" * 100)
print("COMPREHENSIVE AGENT EVALUATION")
print("=" * 100)

# Select a complex query for evaluation
eval_query = "Which team has won the most matches and what rules should they fc
print(f"\nQuery: {eval_query}\n")

# Run agent and capture details
print("Running agent...")
eval_answer = volleyball_agent.run(eval_query)

# Mock data for demonstration (in practice, extract from agent execution)
eval_tools_used = ['volleyball_statistics_analyzer', 'volleyball_knowledge_sear
eval_reasoning = "Agent reasoning trace would be captured here..."
eval_context = "Retrieved context would be captured here..."

# Perform comprehensive evaluation
print("\n" + "-" * 100)
print("EVALUATION RESULTS:")
print("-" * 100)

eval_results = comprehensive_volleyball_agent_evaluation(
    query=eval_query,
    answer=eval_answer,
    tools_used=eval_tools_used,
    reasoning_trace=eval_reasoning,
    context=eval_context
)

for metric, result in eval_results.items():
    print(f"\n{'='*100}")
    print(f"{metric.upper().replace('_', ' ')}")
    print(f"{'='*100}")
    print(result)
```

Summary of Notebook 2 Challenges

New Challenges (Beyond Notebook 1):

7. ✅ **CSV Analyzer Tool** - Statistical analysis with pandas
8. ✅ **Rules Comparison Tool** - Multi-document reasoning
9. ✅ **Multi-Tool Agent** - Tool selection and orchestration
10. ✅ **Complex Query Handling** - Test diverse query types
11. ✅ **Agent Performance Evaluation** - Tool usage and reasoning quality assessment

Learning Outcomes:

- Building domain-specific tools
- Multi-tool agent orchestration
- Reasoning transparency with verbose mode
- Agent-specific evaluation metrics
- Complex multi-step query handling

📕 Notebook 3: Advanced Agentic RAG with VolleyBall Dataset (LangChain)

Filename: `Advance_Agentic_RAG_with_Langchain_VolleyBall.ipynb`

Reference: Based on `Advance_Agentic_RAG_with_Langchain.ipynb`

Objectives

- Implement all advanced RAG techniques with volleyball data
- Progressive agent enhancement (V1 through V5)
- Advanced CSV integration with pandas
- Cross-document reasoning
- Production-ready implementation

Additional Challenges Beyond Notebook 2

Sections 1-7: Base Setup

Instructions:

- Same as Notebook 1 & 2: Installation, imports, models, document loading

---

Section 8: **NEW - Enhanced CSV Tool with Pandas Integration**

🎯 **Challenge 12: Advanced Statistical Analysis Tool**

Instructions:

Create an advanced CSV analysis tool integrated with LangChain:

```python
from langchain_classic.agents import Tool

def create_advanced_csv_analyzer(csv_path: str) -> Tool:
    """
    Create advanced CSV analyzer with pandas operations.

    Capabilities:
    - Complex queries (filtering, grouping, aggregation)
    - Time series analysis
    - Team performance trends
    - Win/loss streaks
    - Statistical comparisons
    - Data visualization descriptions

    Args:
        csv_path: Path to volleyball matches CSV

    Returns:
        LangChain Tool for CSV analysis
    """

    df = pd.read_csv(csv_path)

    def analyze_csv(query: str) -> str:
        """
        Advanced CSV analysis based on query.

        Supports:
        - Aggregations: "average score", "total wins"
        - Filtering: "matches in August", "games by Team A"
        - Trends: "performance over time", "win streaks"
        - Comparisons: "Team A vs Team B statistics"
        """
        try:
            query_lower = query.lower()
            result = f"Analysis: {query}\n\n"

            # TODO: Implement advanced pandas operations

            # Example 1: Time series analysis
            if 'trend' in query_lower or 'over time' in query_lower:
                # Group by date/month and analyze trends
                result += "Time Series Analysis:\n"
                # Add trend analysis code...

            # Example 2: Team comparisons
            elif 'compare' in query_lower or 'vs' in query_lower:
                # Compare team statistics side by side
                result += "Comparative Analysis:\n"
                # Add comparison code...

            # Example 3: Aggregation queries
```

```
            # Example 3: Aggregation queries
            elif 'average' in query_lower or 'mean' in query_lower or 'total' i
                # Calculate aggregates
                result += "Aggregated Statistics:\n"
                # Add aggregation code...

            # Example 4: Filtering queries
            elif 'where' in query_lower or 'when' in query_lower or 'which' in
                # Filter and display subset
                result += "Filtered Results:\n"
                # Add filtering code...

            # Default: Show comprehensive stats
            else:
                result += "Comprehensive Statistics:\n"
                result += f"- Total matches: {len(df)}\n"
                result += f"- Date range: {df['date'].min()} to {df['date'].ma
                result += f"\nSample data:\n{df.head().to_string()}\n"

            return result

        except Exception as e:
            return f"Error in analysis: {str(e)}"

    return Tool(
        name="advanced_volleyball_statistics",
        func=analyze_csv,
        description="Advanced statistical analysis of volleyball match data wit
    )


# Create advanced CSV tool
advanced_csv_tool = create_advanced_csv_analyzer(
    "../UnstructuredDataVolleyBall/CSVFiles/vb_matches.csv"
)

print("✓ Advanced CSV analyzer tool created!")
```

Section 9: **NEW - Agent V1: Basic Retriever**

Instructions:

Implement baseline agent with simple retrieval:

```python
# Create basic retriever tool
def basic_volleyball_retriever(query: str) -> str:
    """Basic vector similarity search"""
    docs = vectorstore.similarity_search(query, k=5)

    result = f"Retrieved {len(docs)} documents:\n\n"
    for i, doc in enumerate(docs, 1):
        result += f"Doc {i} [{doc.metadata.get('source_type')}]: "
        result += f"{doc.page_content[:200]}...\n\n"

    return result

basic_retriever_tool = Tool(
    name="volleyball_retriever_v1",
    func=basic_volleyball_retriever,
    description="Retrieve volleyball information using basic vector similarity"
)

# Create Agent V1
agent_v1_executor = AgentExecutor(
    agent=create_react_agent(llm, [basic_retriever_tool], react_prompt),
    tools=[basic_retriever_tool],
    verbose=True,
    max_iterations=5
)

print("✓ Agent V1 created (Basic Retriever)")
```

Section 10: **NEW - Agent V2: With Hypothetical Questions**

🎯 Challenge 13: Volleyball-Specific Question Generation

Instructions:

Generate volleyball-domain hypothetical questions:

```python
def generate_volleyball_hypothetical_questions(query: str, num: int = 2) -> Lis
    """
    Generate volleyball-related hypothetical questions.

    Considers:
    - Different volleyball formats (beach, indoor, etc.)
    - Related rule areas
    - Statistical angles
    - Coaching/educational perspectives
    """
    prompt = f"""Given this volleyball question, generate {num} related questic
that would help provide comprehensive context.

Original Question: {query}

Consider questions about:
- Related rules or regulations
- Different volleyball formats
- Statistical data that might be relevant
- Coaching or technique aspects
- Historical context or examples

Generate {num} related questions:
1."""

    response = llm.invoke(prompt)

    # Parse questions
    lines = response.strip().split('\n')
    questions = []
    for line in lines:
        clean_line = re.sub(r'^\d+\.\s*', '', line.strip())
        if clean_line and len(clean_line) > 10:
            questions.append(clean_line)

    return questions[:num]


def retriever_with_volleyball_hypotheticals(query: str) -> str:
    """Retrieve using original query + volleyball-specific hypothetical questic

    # Generate related questions
    related_questions = generate_volleyball_hypothetical_questions(query, 2)

    print(f"Original Query: {query}")
    print(f"Generated Questions:")
    for i, q in enumerate(related_questions, 1):
        print(f"  {i}. {q}")

    # Retrieve for all questions
    all_docs = []
    seen = set()
```

```
    seen = set()

    for q in [query] + related_questions:
        docs = vectorstore.similarity_search(q, k=3)
        for doc in docs:
            doc_hash = hash(doc.page_content)
            if doc_hash not in seen:
                seen.add(doc_hash)
                all_docs.append(doc)

    # Format results
    result = f"Retrieved {len(all_docs)} unique documents:\n\n"
    for i, doc in enumerate(all_docs[:7], 1):
        result += f"Doc {i} [{doc.metadata.get('source_type')} - {doc.metadata.
        result += f"{doc.page_content[:250]}...\n\n"

    return result

hypothetical_tool_v2 = Tool(
    name="volleyball_retriever_v2",
    func=retriever_with_volleyball_hypotheticals,
    description="Retrieve volleyball information using hypothetical question ex
)

# Create Agent V2
agent_v2_executor = AgentExecutor(
    agent=create_react_agent(llm, [hypothetical_tool_v2], react_prompt),
    tools=[hypothetical_tool_v2],
    verbose=True,
    max_iterations=10
)

print("✓ Agent V2 created (+ Hypothetical Questions)")
```

Section 11: **NEW - Agent V3: With Hybrid Search**

🎯 Challenge 14: BM25 + Vector for Volleyball Data

Instructions:

Implement hybrid search optimized for volleyball terminology:

```python
from langchain_community.retrievers import BM25Retriever
from langchain_classic.retrievers import EnsembleRetriever

# Create BM25 retriever (keyword-based)
bm25_retriever = BM25Retriever.from_documents(all_documents)
bm25_retriever.k = 5

# Add volleyball-specific keywords boost
volleyball_keywords = [
    'serve', 'spike', 'dig', 'set', 'block', 'rotation',
    'libero', 'rally', 'side-out', 'net', 'court', 'match',
    'beach', 'indoor', 'sand', 'grass'
]

# Create vector retriever
vector_retriever = vectorstore.as_retriever(search_kwargs={'k': 5})

# Create ensemble (hybrid) retriever
hybrid_retriever_v3 = EnsembleRetriever(
    retrievers=[bm25_retriever, vector_retriever],
    weights=[0.4, 0.6]  # 40% keyword, 60% semantic
)

def volleyball_hybrid_search(query: str) -> str:
    """Hybrid search combining keyword and semantic retrieval"""

    docs = hybrid_retriever_v3.invoke(query)

    result = f"Hybrid Search Results ({len(docs)} documents):\n\n"
    for i, doc in enumerate(docs, 1):
        result += f"Doc {i} [{doc.metadata.get('source_type')} - {doc.metadata.
        result += f"Content: {doc.page_content[:200]}...\n\n"

    return result

hybrid_tool_v3 = Tool(
    name="volleyball_retriever_v3",
    func=volleyball_hybrid_search,
    description="Hybrid volleyball retrieval combining keyword (BM25) and semar
)

# Create Agent V3
agent_v3_executor = AgentExecutor(
    agent=create_react_agent(llm, [hybrid_tool_v3], react_prompt),
    tools=[hybrid_tool_v3],
    verbose=True,
    max_iterations=10
)

print("✓ Agent V3 created (+ Hybrid Search)")
```

Key Benefits for Volleyball Domain:

- BM25 captures exact volleyball terminology
- Vector search handles semantic questions
- Combined approach reduces missed relevant documents
- Better for domain-specific jargon

Section 12: **NEW - Agent V4: With Cross-Encoder Re-Ranking**

🎯 **Challenge 15: Volleyball-Optimized Re-Ranking**

Instructions:

Add cross-encoder re-ranking for improved relevance:

```python
from sentence_transformers import CrossEncoder

# Load cross-encoder
cross_encoder = CrossEncoder('cross-encoder/ms-marco-MiniLM-L-6-v2')

def volleyball_retrieval_with_reranking(query: str, top_k: int = 5) -> str:
    """
    Retrieve with hybrid search and re-rank with cross-encoder.

    Process:
    1. Hybrid retrieval (get more candidates, e.g., 10)
    2. Cross-encoder scoring of query-document pairs
    3. Re-rank by score
    4. Return top k
    """

    # Step 1: Retrieve more candidates
    initial_docs = hybrid_retriever_v3.invoke(query)[:10]

    if not initial_docs:
        return "No documents retrieved."

    # Step 2: Score with cross-encoder
    pairs = [[query, doc.page_content] for doc in initial_docs]
    scores = cross_encoder.predict(pairs)

    # Step 3: Re-rank
    ranked = sorted(zip(initial_docs, scores), key=lambda x: x[1], reverse=True
    top_docs = [doc for doc, score in ranked[:top_k]]

    # Format results with scores
    result = f"Re-ranked Results (top {top_k} of {len(initial_docs)}):\n\n"
    for i, (doc, score) in enumerate(ranked[:top_k], 1):
        result += f"Doc {i} [Score: {score:.3f}] [{doc.metadata.get('source_typ
        result += f"{doc.page_content[:200]}...\n\n"

    return result

reranking_tool_v4 = Tool(
    name="volleyball_retriever_v4",
    func=volleyball_retrieval_with_reranking,
    description="Advanced retrieval with hybrid search and cross-encoder re-ran
)

# Create Agent V4
agent_v4_executor = AgentExecutor(
    agent=create_react_agent(llm, [reranking_tool_v4], react_prompt),
    tools=[reranking_tool_v4],
    verbose=True,
    max_iterations=10
)
```

```
print("✓ Agent V4 created (+ Cross-Encoder Re-Ranking)")
```

Expected Improvements:

- 15-30% better precision
- More relevant top results
- Better handling of ambiguous queries

Section 13: **NEW - Agent V5: With LLM Compression**

🎯 Challenge 16: Volleyball-Focused Context Compression

Instructions:

Add LLM-based contextual compression to extract only volleyball-relevant information:

```python
from langchain_classic.retrievers.document_compressors import LLMChainExtractor
from langchain_classic.retrievers import ContextualCompressionRetriever

# Create compressor
compressor = LLMChainExtractor.from_llm(llm)

# Create compression retriever (wraps hybrid retriever)
compression_retriever_v5 = ContextualCompressionRetriever(
    base_compressor=compressor,
    base_retriever=hybrid_retriever_v3
)

def volleyball_compressed_retrieval(query: str) -> str:
    """
    Retrieve and compress context using LLM.

    Benefits:
    - Removes irrelevant information
    - Focuses on query-specific content
    - Reduces token usage
    - Improves LLM focus
    """

    print(f"Retrieving and compressing for: {query}")

    # Retrieve with compression
    compressed_docs = compression_retriever_v5.invoke(query)

    if not compressed_docs:
        return "No relevant information found."

    result = f"Compressed Results ({len(compressed_docs)} documents):\n\n"
    for i, doc in enumerate(compressed_docs, 1):
        # Compressed content should be significantly shorter
        result += f"Doc {i} [Compressed from {doc.metadata.get('source_type')}]
        result += f"{doc.page_content}\n\n"

    return result

compression_tool_v5 = Tool(
    name="volleyball_retriever_v5",
    func=volleyball_compressed_retrieval,
    description="Advanced retrieval with hybrid search, re-ranking, and LLM com
)

# Create Agent V5 (Final, most advanced)
agent_v5_executor = AgentExecutor(
    agent=create_react_agent(llm, [compression_tool_v5, advanced_csv_tool], rea
    tools=[compression_tool_v5, advanced_csv_tool],
    verbose=True,
    max_iterations=10
)
```

```
,

print("✓ Agent V5 created (+ LLM Contextual Compression)")
print("✓ Agent V5 also includes advanced CSV analyzer for comprehensive capabi
```

Section 14: **NEW - Comparative Agent Performance Testing**

🎯 Challenge 17: Progressive Performance Comparison

Instructions:

Test all 5 agent versions with the same queries and compare:

```python
# Define comprehensive test queries
volleyball_test_queries = [
    {
        'query': "What are the serving rules in beach volleyball?",
        'type': 'rule_specific',
        'complexity': 'simple'
    },
    {
        'query': "What are the differences between beach and grass volleyball s
        'type': 'comparative',
        'complexity': 'medium'
    },
    {
        'query': "Based on match statistics, what serving strategies seem most
        'type': 'multi_source',
        'complexity': 'complex'
    },
]

def compare_agent_performance(query_dict: dict):
    """
    Test query across all agent versions and compare.

    Compares:
    - Retrieval quality
    - Answer completeness
    - Response time
    - Context relevance
    """
    query = query_dict['query']

    print(f"\n{'='*100}")
    print(f"TESTING QUERY: {query}")
    print(f"Type: {query_dict['type']} | Complexity: {query_dict['complexity']}
    print(f"{'='*100}")

    results = {}

    # Test V1
    print(f"\n{'→'*50}")
    print("AGENT V1 (Basic Retriever)")
    print(f"{'→'*50}")
    try:
        result_v1 = agent_v1_executor.invoke({"input": query})
        results['v1'] = result_v1['output']
    except Exception as e:
        results['v1'] = f"Error: {e}"

    # Test V2
    print(f"\n{'→'*50}")
    print("AGENT V2 (+ Hypothetical Questions)")
    print(f"{'→'*50}")
```

```python
    try:
        result_v2 = agent_v2_executor.invoke({"input": query})
        results['v2'] = result_v2['output']
    except Exception as e:
        results['v2'] = f"Error: {e}"

    # Test V3
    print(f"\n{'→'*50}")
    print("AGENT V3 (+ Hybrid Search)")
    print(f"{'→'*50}")
    try:
        result_v3 = agent_v3_executor.invoke({"input": query})
        results['v3'] = result_v3['output']
    except Exception as e:
        results['v3'] = f"Error: {e}"

    # Test V4
    print(f"\n{'→'*50}")
    print("AGENT V4 (+ Re-Ranking)")
    print(f"{'→'*50}")
    try:
        result_v4 = agent_v4_executor.invoke({"input": query})
        results['v4'] = result_v4['output']
    except Exception as e:
        results['v4'] = f"Error: {e}"

    # Test V5
    print(f"\n{'→'*50}")
    print("AGENT V5 (+ Compression + CSV)")
    print(f"{'→'*50}")
    try:
        result_v5 = agent_v5_executor.invoke({"input": query})
        results['v5'] = result_v5['output']
    except Exception as e:
        results['v5'] = f"Error: {e}"

    # Display comparison
    print(f"\n{'='*100}")
    print("COMPARISON RESULTS")
    print(f"{'='*100}")

    for version, answer in results.items():
        print(f"\n--- {version.upper()} ANSWER ---")
        print(answer[:300] + "..." if len(answer) > 300 else answer)
        print("-" * 100)

    return results


# Run comparisons
comparison_results = {}
```

```
for test_query in volleyball_test_queries:
    comparison_results[test_query['query']] = compare_agent_performance(test_qu
```

Section 14B: **NEW - KPI-Based Agent Evaluation & Comparison**

🎯 Challenge 17B: Evaluate Quality Metrics & Performance KPIs

Instructions:

Implement comprehensive evaluation comparing three key agents from the Advanced RAG pipeline:

- **Agent V1** (Basic Retriever)
- **Agent V3** (Hybrid Search)
- **Agent V5** (Compression + CSV)

Evaluate using quality metrics and performance KPIs:

Quality Metrics:

1. Groundedness - Answer supported by context
2. Context Relevance - Retrieved context relevance to question
3. Answer Relevance - Answer relevance to question

Performance KPI: 4. Execution Time - Response time in seconds

Implementation Template:

```python
import time
from typing import Dict, List, Tuple
import pandas as pd

# Define evaluation test queries (diverse complexity)
evaluation_test_queries = [
    {
        'query': "What are the serving rules in beach volleyball?",
        'type': 'rule_lookup',
        'complexity': 'simple'
    },
    {
        'query': "How do scoring systems differ between beach and indoor volley
        'type': 'comparative',
        'complexity': 'medium'
    },
    {
        'query': "Which team has the best performance and what training drills
        'type': 'multi_source_analysis',
        'complexity': 'complex'
    },
    {
        'query': "What are the court dimensions for grass volleyball?",
        'type': 'specific_fact',
        'complexity': 'simple'
    },
    {
        'query': "Based on match data, what are common faults and what do the r
        'type': 'data_rule_correlation',
        'complexity': 'complex'
    }
]

def evaluate_groundedness(question: str, context: str, answer: str, llm) -> Dic
    """
    Evaluate if the answer is grounded in the provided context.

    Checks:
    - Are claims in the answer supported by context?
    - Are there unsupported assertions?
    - Does answer hallucinate information?

    Args:
        question: The user's question
        context: Retrieved context used for generation
        answer: Generated answer
        llm: Language model for evaluation

    Returns:
        Dictionary with score and explanation
    """
    evaluation_prompt = f"""You are an expert evaluator assessing answer groun
```

```
    evaluation_prompt = f"""You are an expert evaluator assessing answer ground

**Context Provided**:
{context[:2000]}... [truncated if longer]

**Question**: {question}

**Answer**: {answer}

**Task**: Evaluate if the answer is fully grounded in the provided context.

**Evaluation Criteria** (1-5 scale):
1 - Completely ungrounded: Answer contains information not in context, hallucin
2 - Mostly ungrounded: Some grounded claims but significant unsupported content
3 - Partially grounded: Mix of grounded and unsupported claims
4 - Mostly grounded: Almost all claims supported, minor unsupported details
5 - Fully grounded: Every claim is directly supported by the context

**Required Output Format**:
Score: [1-5]
Grounded Claims: [List 2-3 claims that ARE supported]
Ungrounded Claims: [List any claims that are NOT supported, or "None"]
Reasoning: [2-3 sentences explaining the score]
"""

    response = llm(evaluation_prompt, max_new_tokens=400, temperature=0.1)
    result_text = response[0]['generated_text'][len(evaluation_prompt):].strip(

    # Parse score from response
    score = 0.0
    try:
        for line in result_text.split('\n'):
            if line.startswith('Score:'):
                score = float(line.split(':')[1].strip().split()[0])
                break
    except:
        score = 3.0  # Default if parsing fails

    return {
        'score': score,
        'explanation': result_text,
        'metric': 'groundedness'
    }


def evaluate_context_relevance(question: str, context: str, llm) -> Dict[str, a
    """
    Evaluate if the retrieved context is relevant to answering the question.

    Checks:
    - Does context contain information needed to answer?
    - Is context on-topic?
```

```
    - How much of context is relevant vs. noise?

    Args:
        question: The user's question
        context: Retrieved context
        llm: Language model for evaluation

    Returns:
        Dictionary with score and explanation
    """
    evaluation_prompt = f"""You are an expert evaluator assessing context relev

**Question**: {question}

**Retrieved Context**:
{context[:2000]}... [truncated if longer]

**Task**: Evaluate if the retrieved context is relevant for answering the quest

**Evaluation Criteria** (1-5 scale):
1 - Completely irrelevant: Context has no relation to the question
2 - Mostly irrelevant: Little useful information for answering
3 - Partially relevant: Some useful information but much noise
4 - Mostly relevant: Context contains most information needed
5 - Highly relevant: Context is perfectly targeted for the question

**Required Output Format**:
Score: [1-5]
Relevant Information: [What relevant info is present]
Missing Information: [What's needed but missing, or "None"]
Noise Level: [Low/Medium/High - how much irrelevant content]
Reasoning: [2-3 sentences explaining the score]
"""

    response = llm(evaluation_prompt, max_new_tokens=400, temperature=0.1)
    result_text = response[0]['generated_text'][len(evaluation_prompt):].strip(

    # Parse score
    score = 0.0
    try:
        for line in result_text.split('\n'):
            if line.startswith('Score:'):
                score = float(line.split(':')[1].strip().split()[0])
                break
    except:
        score = 3.0

    return {
        'score': score,
        'explanation': result_text,
        'metric': 'context_relevance'
    }
```

```
def evaluate_answer_relevance(question: str, answer: str, llm) -> Dict[str, any
    """
    Evaluate if the answer is relevant to the question asked.

    Checks:
    - Does answer address the question?
    - Is answer on-topic?
    - Does answer provide what was asked for?

    Args:
        question: The user's question
        answer: Generated answer
        llm: Language model for evaluation

    Returns:
        Dictionary with score and explanation
    """
    evaluation_prompt = f"""You are an expert evaluator assessing answer releva

**Question**: {question}

**Answer**: {answer}

**Task**: Evaluate if the answer is relevant and responsive to the question.

**Evaluation Criteria** (1-5 scale):
1 - Completely irrelevant: Answer doesn't address the question at all
2 - Mostly irrelevant: Answer is largely off-topic
3 - Partially relevant: Answer touches on question but misses key aspects
4 - Mostly relevant: Answer addresses most aspects of the question
5 - Highly relevant: Answer directly and fully addresses the question

**Required Output Format**:
Score: [1-5]
What Was Asked: [Summarize what question wanted]
What Was Provided: [Summarize what answer provided]
Gaps: [What's missing from answer, or "None"]
Reasoning: [2-3 sentences explaining the score]
"""

    response = llm(evaluation_prompt, max_new_tokens=400, temperature=0.1)
    result_text = response[0]['generated_text'][len(evaluation_prompt):].strip(

    # Parse score
    score = 0.0
    try:
        for line in result_text.split('\n'):
            if line.startswith('Score:'):
                score = float(line.split(':')[1].strip().split()[0])
                break
```

```
                break
    except:
        score = 3.0

    return {
        'score': score,
        'explanation': result_text,
        'metric': 'answer_relevance'
    }


def evaluate_agent_with_kpis(
    agent_executor,
    agent_name: str,
    query: str,
    retriever,
    llm
) -> Dict[str, any]:
    """
    Comprehensive evaluation of an agent with quality metrics and performance K

    Evaluates:
    - Groundedness (quality)
    - Context Relevance (quality)
    - Answer Relevance (quality)
    - Execution Time (performance)

    Args:
        agent_executor: The agent to evaluate
        agent_name: Name/version of agent (e.g., "V1_Basic")
        query: Test query
        retriever: Retriever for getting context
        llm: LLM for evaluation

    Returns:
        Dictionary with all metrics
    """
    print(f"\n{'='*80}")
    print(f"Evaluating {agent_name}: {query[:60]}...")
    print(f"{'='*80}")

    results = {
        'agent': agent_name,
        'query': query
    }

    # Measure execution time
    start_time = time.time()

    try:
        # Execute agent
        agent_response = agent_executor.invoke({"input": query})
```

```
        answer = agent_response['output']
        execution_time = time.time() - start_time

        results['answer'] = answer
        results['execution_time'] = round(execution_time, 2)
        results['status'] = 'success'

        print(f"✓ Agent responded in {execution_time:.2f}s")

        # Retrieve context for evaluation
        print("  → Retrieving context for evaluation...")
        context_docs = retriever.invoke(query)
        context = "\n\n".join([doc.page_content for doc in context_docs])

        # Evaluate quality metrics
        print("  → Evaluating Groundedness...")
        groundedness_eval = evaluate_groundedness(query, context, answer, llm)
        results['groundedness_score'] = groundedness_eval['score']
        results['groundedness_detail'] = groundedness_eval['explanation']

        print("  → Evaluating Context Relevance...")
        context_rel_eval = evaluate_context_relevance(query, context, llm)
        results['context_relevance_score'] = context_rel_eval['score']
        results['context_relevance_detail'] = context_rel_eval['explanation']

        print("  → Evaluating Answer Relevance...")
        answer_rel_eval = evaluate_answer_relevance(query, answer, llm)
        results['answer_relevance_score'] = answer_rel_eval['score']
        results['answer_relevance_detail'] = answer_rel_eval['explanation']

        # Calculate average quality score
        avg_quality = round(
            (groundedness_eval['score'] +
             context_rel_eval['score'] +
             answer_rel_eval['score']) / 3,
            2
        )
        results['average_quality_score'] = avg_quality

        print(f"✓ Evaluation complete - Avg Quality: {avg_quality}/5.0")

    except Exception as e:
        execution_time = time.time() - start_time
        results['answer'] = f"Error: {str(e)}"
        results['execution_time'] = round(execution_time, 2)
        results['status'] = 'error'
        results['groundedness_score'] = 0.0
        results['context_relevance_score'] = 0.0
        results['answer_relevance_score'] = 0.0
        results['average_quality_score'] = 0.0
        print(f"✗ Agent failed after {execution_time:.2f}s: {e}")
```

```
    return results


def compare_agents_comprehensive(
    test_queries: List[Dict],
    agent_configs: List[Tuple],
    llm
) -> pd.DataFrame:
    """
    Compare multiple agents across test queries with full KPI evaluation.

    Args:
        test_queries: List of test query dictionaries
        agent_configs: List of (agent_executor, agent_name, retriever) tuples
        llm: LLM for evaluation

    Returns:
        DataFrame with comprehensive comparison results
    """
    all_results = []

    print("\n" + "=" * 100)
    print("COMPREHENSIVE AGENT COMPARISON WITH KPI EVALUATION")
    print("=" * 100)
    print(f"Testing {len(agent_configs)} agents on {len(test_queries)} queries'
    print(f"Metrics: Groundedness, Context Relevance, Answer Relevance, Executi
    print("=" * 100)

    for query_dict in test_queries:
        query = query_dict['query']
        query_type = query_dict['type']
        complexity = query_dict['complexity']

        print(f"\n{'▓'*100}")
        print(f"QUERY: {query}")
        print(f"Type: {query_type} | Complexity: {complexity}")
        print(f"{'▓'*100}")

        for agent_executor, agent_name, retriever in agent_configs:
            result = evaluate_agent_with_kpis(
                agent_executor=agent_executor,
                agent_name=agent_name,
                query=query,
                retriever=retriever,
                llm=llm
            )
            result['query_type'] = query_type
            result['complexity'] = complexity
            all_results.append(result)

    # Create DataFrame
    df = pd.DataFrame(all_results)
```

```
    df = pd.DataFrame(all_results)

    return df


# Configure agents for comparison
agent_comparison_configs = [
    (agent_v1_executor, "V1_Basic", vectorstore.as_retriever(search_kwargs={'k'
    (agent_v3_executor, "V3_Hybrid_Search", hybrid_retriever_v3),
    (agent_v5_executor, "V5_Compression", compression_retriever_v5),
]

# Run comprehensive comparison
print("\n" + "🎯" * 50)
print("Starting KPI-Based Agent Evaluation...")
print("🎯" * 50)

comparison_df = compare_agents_comprehensive(
    test_queries=evaluation_test_queries,
    agent_configs=agent_comparison_configs,
    llm=llm_pipeline
)

# Display results
print("\n" + "=" * 100)
print("EVALUATION RESULTS SUMMARY")
print("=" * 100)

# Summary statistics by agent
print("\n📊 AVERAGE SCORES BY AGENT:")
print("-" * 100)

summary_by_agent = comparison_df.groupby('agent').agg({
    'groundedness_score': 'mean',
    'context_relevance_score': 'mean',
    'answer_relevance_score': 'mean',
    'average_quality_score': 'mean',
    'execution_time': 'mean'
}).round(2)

print(summary_by_agent.to_string())

# Performance vs Quality Trade-off
print("\n⚖️ PERFORMANCE VS QUALITY TRADE-OFF:")
print("-" * 100)

tradeoff_summary = summary_by_agent[['average_quality_score', 'execution_time']
tradeoff_summary['quality_per_second'] = (
    tradeoff_summary['average_quality_score'] / tradeoff_summary['execution_tim
).round(3)

print(tradeoff_summary.to_string())
```

```
# Best performer identification
best_quality = summary_by_agent['average_quality_score'].idxmax()
fastest = summary_by_agent['execution_time'].idxmin()
best_tradeoff = tradeoff_summary['quality_per_second'].idxmax()

print(f"\n🏆 BEST PERFORMERS:")
print(f"  • Highest Quality: {best_quality} ({summary_by_agent.loc[best_quality
print(f"  • Fastest: {fastest} ({summary_by_agent.loc[fastest, 'execution_time'
print(f"  • Best Quality/Speed Ratio: {best_tradeoff} ({tradeoff_summary.loc[be

# Results by query complexity
print("\n📈 PERFORMANCE BY QUERY COMPLEXITY:")
print("-" * 100)

complexity_analysis = comparison_df.groupby(['complexity', 'agent']).agg({
    'average_quality_score': 'mean',
    'execution_time': 'mean'
}).round(2)

print(complexity_analysis.to_string())

# Detailed results table
print("\n📋 DETAILED RESULTS TABLE:")
print("-" * 100)

detailed_view = comparison_df[[
    'agent', 'query_type', 'complexity',
    'groundedness_score', 'context_relevance_score', 'answer_relevance_score',
    'average_quality_score', 'execution_time', 'status'
]]

print(detailed_view.to_string(index=False))

# Save results to CSV for further analysis
output_path = "./agent_comparison_results.csv"
comparison_df.to_csv(output_path, index=False)
print(f"\n💾 Full results saved to: {output_path}")

# Visualization recommendations
print("\n" + "=" * 100)
print("📊 VISUALIZATION RECOMMENDATIONS")
print("=" * 100)
print("""
To visualize these results, consider creating:

1. **Bar Chart - Average Quality Scores by Agent**
   • X-axis: Agent versions (V1, V3, V5)
   • Y-axis: Quality score (0-5)
   • Grouped bars for: Groundedness, Context Relevance, Answer Relevance

2. **Scatter Plot - Quality vs Speed Trade-off**
```

2. **Scatter Plot - Quality vs Speed Trade-off**
   - X-axis: Execution Time (seconds)
   - Y-axis: Average Quality Score
   - Points: Each agent (V1, V3, V5)
   - Ideal: Top-left corner (high quality, low time)

3. **Heatmap - Performance by Complexity**
   - Rows: Query complexity (simple, medium, complex)
   - Columns: Agents (V1, V3, V5)
   - Color: Quality score or execution time

4. **Line Chart - Quality Metrics Comparison**
   - X-axis: Agents (V1 → V3 → V5)
   - Y-axis: Score (0-5)
   - Lines: Groundedness, Context Relevance, Answer Relevance
   - Shows progression across agent versions

Example code for visualization (using matplotlib/seaborn):

```python
import matplotlib.pyplot as plt
import seaborn as sns

# 1. Quality Metrics Comparison
fig, ax = plt.subplots(figsize=(12, 6))
summary_by_agent[['groundedness_score', 'context_relevance_score', 'answer_rele
    kind='bar', ax=ax, rot=0
)
ax.set_title('Quality Metrics Comparison Across Agents', fontsize=14, fontweigh
ax.set_ylabel('Score (1-5)', fontsize=12)
ax.set_xlabel('Agent Version', fontsize=12)
ax.legend(['Groundedness', 'Context Relevance', 'Answer Relevance'])
ax.grid(axis='y', alpha=0.3)
plt.tight_layout()
plt.savefig('quality_comparison.png', dpi=300)
plt.show()

# 2. Quality vs Speed Trade-off
fig, ax = plt.subplots(figsize=(10, 6))
for agent in summary_by_agent.index:
    ax.scatter(
        summary_by_agent.loc[agent, 'execution_time'],
        summary_by_agent.loc[agent, 'average_quality_score'],
        s=200, alpha=0.7, label=agent
    )
    ax.annotate(
        agent,
        (summary_by_agent.loc[agent, 'execution_time'],
         summary_by_agent.loc[agent, 'average_quality_score']),
        fontsize=10, fontweight='bold'
    )
ax.set_title('Quality vs Speed Trade-off', fontsize=14, fontweight='bold')
```

```
ax.set_xlabel('Execution Time (seconds)', fontsize=12)
ax.set_ylabel('Average Quality Score (1-5)', fontsize=12)
ax.grid(True, alpha=0.3)
ax.legend()
plt.tight_layout()
plt.savefig('quality_vs_speed.png', dpi=300)
plt.show()

# 3. Performance by Complexity Heatmap
pivot_quality = comparison_df.pivot_table(
    values='average_quality_score',
    index='complexity',
    columns='agent',
    aggfunc='mean'
)

fig, ax = plt.subplots(figsize=(10, 6))
sns.heatmap(pivot_quality, annot=True, fmt='.2f', cmap='YlGnBu', ax=ax, vmin=0,
ax.set_title('Average Quality Score by Query Complexity and Agent', fontsize=14
ax.set_xlabel('Agent Version', fontsize=12)
ax.set_ylabel('Query Complexity', fontsize=12)
plt.tight_layout()
plt.savefig('complexity_heatmap.png', dpi=300)
plt.show()

print("✓ Visualizations saved!")
```

""")

print("\n" + "=" * 100) print("KEY INSIGHTS & RECOMMENDATIONS") print("=" * 100) print("""
Expected Observations:

1. **V1 (Basic Retriever)** • Strengths: Fastest execution, simple implementation • Weaknesses: May miss relevant context, lower quality scores • Best for: Simple queries, low-latency requirements
2. **V3 (Hybrid Search)** • Strengths: Better recall, combines keyword + semantic • Weaknesses: Slower than V1, may retrieve more noise • Best for: Queries with specific terminology, medium complexity
3. **V5 (Compression)** • Strengths: Highest quality, focused context, best groundedness • Weaknesses: Slowest due to compression step • Best for: Complex queries, quality-critical applications

Production Recommendations: • Use V1 for real-time, simple lookups • Use V3 as default for balanced performance • Use V5 for complex analysis or quality-critical scenarios • Consider adaptive routing based on query complexity """)

print("\n✓ KPI-Based Agent Evaluation Complete!") print("=" * 100)

```
**Key Concepts:**

1. **Evaluation Framework:**
   - Each agent is tested with identical queries
   - Measures both quality (groundedness, relevance) and performance (time)
   - Provides quantitative comparison basis

2. **Quality Metrics:**
   - **Groundedness**: Prevents hallucinations, ensures factual accuracy
   - **Context Relevance**: Measures retrieval effectiveness
   - **Answer Relevance**: Ensures response addresses the question

3. **Performance KPI:**
   - **Execution Time**: Critical for production systems
   - Trade-off analysis: quality vs. speed
   - Quality-per-second metric for efficiency

4. **Comparative Analysis:**
   - By agent: Which performs best overall?
   - By complexity: How do agents handle different difficulty levels?
   - Trade-offs: When to use which agent?

5. **Production Insights:**
   - Data-driven agent selection
   - Understanding performance characteristics
   - Adaptive routing strategies

**Expected Results Pattern:**
```

V1 (Basic): ⚡ Fast | ⭐⭐⭐ Medium Quality | Simple Queries V3 (Hybrid): ⚡⚡ Medium | ⭐⭐⭐⭐ Good Quality | Balanced V5 (Compress): ⚡⚡⚡ Slow | ⭐⭐⭐⭐⭐ Best Quality | Complex Queries

---

Section 15: **NEW - Advanced Cross-Document Reasoning**

🎯 Challenge 18: Synthesize Information Across Sources

Instructions:

Create queries that require combining information from multiple document types:

```python
cross_document_queries = [
    """Based on the match data, identify the team with the best performance.
    Then, review the rules and lesson plans to suggest a training program
    that would help other teams reach that level of performance.""",

    """Analyze match statistics to identify common scoring patterns.
    Then cross-reference with volleyball rules to explain which legal
    techniques are being used most effectively.""",

    """Compare the rules across different volleyball formats (beach, grass, inc
    Then suggest which format might be best for beginners based on lesson plan
    content and rule complexity.""",
]

print("=" * 100)
print("TESTING CROSS-DOCUMENT REASONING")
print("=" * 100)

for i, cross_query in enumerate(cross_document_queries, 1):
    print(f"\n{'='*100}")
    print(f"CROSS-DOCUMENT QUERY {i}:")
    print(f"{'='*100}")
    print(cross_query)
    print(f"\n{'→'*50}")
    print("AGENT V5 PROCESSING (Verbose Mode):")
    print(f"{'→'*50}")

    try:
        result = agent_v5_executor.invoke({"input": cross_query})

        print(f"\n{'='*100}")
        print("SYNTHESIZED ANSWER:")
        print(f"{'='*100}")
        print(result['output'])
        print(f"\n{'='*100}\n")

    except Exception as e:
        print(f"Error: {e}")
```

Key Observation Points:

- How does the agent identify need for multiple sources?
- Tool selection for CSV vs. rules vs. educational content
- Quality of synthesis across different data types
- Coherence of final answer integrating multiple sources

Section 16: **NEW - Metadata-Based Intelligent Filtering**

🎯 **Challenge 19: Dynamic Filter Selection**

Instructions:

Create an agent that intelligently selects metadata filters based on query:

```python
def intelligent_filtered_retrieval(query: str) -> str:
    """
    Intelligently apply metadata filters based on query analysis.

    Query Analysis:
    - Statistical questions → filter to CSV
    - "Beach volleyball" → filter to volleyball_type='beach'
    - "Junior" or "beginner" → filter to educational content
    - "Official rules" → filter to HTML sources
    - Specific organization → filter by organization

    Returns filtered and retrieved documents.
    """
    query_lower = query.lower()
    filters = {}

    # Analyze query for filtering opportunities

    # 1. Volleyball format filtering
    if 'beach' in query_lower:
        filters['volleyball_type'] = 'beach'
    elif 'grass' in query_lower:
        filters['volleyball_type'] = 'grass'
    elif 'sand' in query_lower:
        filters['volleyball_type'] = 'sand'
    elif 'indoor' in query_lower:
        filters['volleyball_type'] = 'indoor'

    # 2. Source type filtering
    if 'statistics' in query_lower or 'match' in query_lower or 'data' in query
        filters['source_type'] = 'csv'
    elif 'official' in query_lower or 'regulation' in query_lower:
        filters['source_type'] = 'html'
    elif 'lesson' in query_lower or 'drill' in query_lower or 'teaching' in que
        filters['document_category'] = 'educational'

    # 3. Competition level filtering
    if 'junior' in query_lower or 'beginner' in query_lower or 'youth' in query
        filters['competition_level'] = 'junior'
    elif 'adult' in query_lower or 'professional' in query_lower:
        filters['competition_level'] = 'adult'

    # Apply filters and retrieve
    filter_desc = ", ".join([f"{k}={v}" for k, v in filters.items()]) if filter
    print(f"Applied filters: {filter_desc}")

    if filters:
        # Use filtered retriever
        filtered_retriever = vectorstore.as_retriever(
            search_type='similarity',
            search_kwargs={'k': 5, 'filter': filters}
        )
```

```
        )
        docs = filtered_retriever.invoke(query)
    else:
        # Use standard retriever
        docs = vectorstore.similarity_search(query, k=5)

    # Format results
    result = f"Intelligent Filtered Retrieval (Filters: {filter_desc}):\n\n"
    result += f"Retrieved {len(docs)} documents:\n\n"

    for i, doc in enumerate(docs, 1):
        result += f"Doc {i}:\n"
        result += f"  Type: {doc.metadata.get('source_type')}\n"
        result += f"  Format: {doc.metadata.get('volleyball_type', 'N/A')}\n"
        result += f"  Category: {doc.metadata.get('document_category', 'N/A')}\
        result += f"  Content: {doc.page_content[:150]}...\n\n"

    return result


intelligent_filter_tool = Tool(
    name="intelligent_filtered_retriever",
    func=intelligent_filtered_retrieval,
    description="Intelligent retrieval with automatic metadata filter selectior
)

# Create agent with intelligent filtering
agent_intelligent_filter = AgentExecutor(
    agent=create_react_agent(llm, [intelligent_filter_tool, advanced_csv_tool],
    tools=[intelligent_filter_tool, advanced_csv_tool],
    verbose=True,
    max_iterations=10
)

print("✓ Intelligent filtering agent created!")

# Test intelligent filtering
filter_test_queries = [
    "What are the official beach volleyball serving rules?",  # Should filter:
    "Show me statistics from junior level matches",           # Should filter:
    "What drills are good for beginner grass volleyball?",    # Should filter:
]

print("\n" + "=" * 100)
print("TESTING INTELLIGENT METADATA FILTERING")
print("=" * 100)

for query in filter_test_queries:
    print(f"\n{'='*100}")
    print(f"Query: {query}")
    print(f"{'='*100}\n")
```

```
    result = agent_intelligent_filter.invoke({"input": query})
    print(f"\nAnswer: {result['output']}")
    print(f"\n{'='*100}")
```

Section 17: **NEW - Production-Ready Error Handling & Monitoring**

🎯 Challenge 20: Robust Production Implementation

Instructions:

Add comprehensive error handling, logging, and monitoring:

```python
import logging
from datetime import datetime
from typing import Optional

# Configure logging
logging.basicConfig(
    level=logging.INFO,
    format='%(asctime)s - %(name)s - %(levelname)s - %(message)s',
    handlers=[
        logging.FileHandler('volleyball_rag.log'),
        logging.StreamHandler()
    ]
)

logger = logging.getLogger('VolleyballRAG')

class ProductionVolleyballRAG:
    """
    Production-ready Volleyball RAG system with comprehensive error handling.

    Features:
    - Error recovery and graceful degradation
    - Query logging and monitoring
    - Performance tracking
    - Fallback strategies
    - Rate limiting
    """

    def __init__(self, agent_executor, vectorstore, llm):
        self.agent = agent_executor
        self.vectorstore = vectorstore
        self.llm = llm
        self.query_count = 0
        self.error_count = 0
        self.query_log = []

    def query(
        self,
        question: str,
        max_retries: int = 2,
        fallback_to_simple: bool = True
    ) -> dict:
        """
        Execute query with comprehensive error handling.

        Args:
            question: User's question
            max_retries: Number of retries on failure
            fallback_to_simple: Whether to fall back to simple RAG on agent fai

        Returns:
            Dictionary with answer, metadata, and status
```

```
        Dictionary with answer, metadata, and status
    """
    start_time = datetime.now()
    self.query_count += 1

    logger.info(f"Query #{self.query_count}: {question}")

    result = {
        'query_id': self.query_count,
        'question': question,
        'timestamp': start_time.isoformat(),
        'status': 'unknown',
        'answer': None,
        'method': None,
        'error': None,
        'latency_ms': None
    }

    # Attempt agent-based query
    for attempt in range(max_retries):
        try:
            logger.info(f"Attempt {attempt + 1}/{max_retries}")

            response = self.agent.invoke({"input": question})

            result['status'] = 'success'
            result['answer'] = response['output']
            result['method'] = 'agent'

            logger.info(f"Success via agent (attempt {attempt + 1})")
            break

        except Exception as e:
            self.error_count += 1
            logger.error(f"Agent error (attempt {attempt + 1}): {str(e)}")
            result['error'] = str(e)

            if attempt == max_retries - 1 and fallback_to_simple:
                # Fallback to simple RAG
                logger.warning("Falling back to simple RAG")

                try:
                    docs = self.vectorstore.similarity_search(question, k=5
                    context = "\n\n".join([doc.page_content for doc in docs

                    prompt = f"Context:\n{context}\n\nQuestion: {question}\
                    answer = self.llm.invoke(prompt)

                    result['status'] = 'success_fallback'
                    result['answer'] = answer
                    result['method'] = 'simple_rag'
```

```
                        logger.info("Success via fallback simple RAG")

                    except Exception as fallback_error:
                        result['status'] = 'failure'
                        result['error'] = f"Agent failed and fallback failed: {
                        logger.error(f"Fallback also failed: {fallback_error}")

        # Calculate latency
        end_time = datetime.now()
        result['latency_ms'] = int((end_time - start_time).total_seconds() * 10

        # Log query
        self.query_log.append(result)

        logger.info(f"Query completed: status={result['status']}, latency={resu

        return result

    def get_statistics(self) -> dict:
        """Get system statistics."""
        return {
            'total_queries': self.query_count,
            'total_errors': self.error_count,
            'error_rate': self.error_count / self.query_count if self.query_cou
            'avg_latency_ms': sum(q['latency_ms'] for q in self.query_log if q[
            'success_rate': sum(1 for q in self.query_log if q['status'].starts
        }


# Create production system
production_rag = ProductionVolleyballRAG(
    agent_executor=agent_v5_executor,
    vectorstore=vectorstore,
    llm=llm
)

print("✓ Production-ready Volleyball RAG system initialized!")

# Test production system
print("\n" + "=" * 100)
print("TESTING PRODUCTION SYSTEM")
print("=" * 100)

test_queries = [
    "What are the serving rules in beach volleyball?",
    "What statistics can you provide about team performance?",
    "This is an intentionally vague query to test error handling"
]

for query in test_queries:
    print(f"\n{'→'*50}")
    print(f"Query: {query}")
```

```
    print(f"{'→'*50}")

    result = production_rag.query(query)

    print(f"\nStatus: {result['status']}")
    print(f"Method: {result['method']}")
    print(f"Latency: {result['latency_ms']}ms")
    print(f"\nAnswer: {result['answer'][:200] if result['answer'] else 'No answ
    print(f"\n{'='*100}")

# Display statistics
print("\n" + "=" * 100)
print("SYSTEM STATISTICS")
print("=" * 100)

stats = production_rag.get_statistics()
for key, value in stats.items():
    print(f"{key}: {value}")
```

Summary of Notebook 3 Challenges

New Challenges (Beyond Notebooks 1 & 2):

12. ✅ **Advanced Statistical Analysis Tool** - Complex pandas operations
13. ✅ **Volleyball-Specific Question Generation** - Domain-aware query expansion
14. ✅ **BM25 + Vector Hybrid** - Optimized for volleyball terminology
15. ✅ **Cross-Encoder Re-Ranking** - Improved relevance scoring
16. ✅ **LLM Contextual Compression** - Focused information extraction
17. ✅ **Progressive Performance Comparison** - Test all 5 agent versions 17B. ✅ **KPI-Based Agent Evaluation** - Quality metrics + performance KPIs comparison (V1, V3, V5)
18. ✅ **Cross-Document Reasoning** - Synthesize from multiple sources
19. ✅ **Intelligent Metadata Filtering** - Dynamic filter selection
20. ✅ **Production-Ready Implementation** - Error handling, logging, monitoring

Learning Outcomes:

- Progressive enhancement of RAG capabilities
- Production-ready error handling and monitoring
- Advanced retrieval techniques (hybrid, re-ranking, compression)
- Cross-document reasoning and synthesis
- Performance optimization and comparison with quantitative metrics
- Quality vs. performance trade-off analysis
- Real-world deployment considerations with data-driven decision making

---

📊 Complete Challenge Summary

All 21 Challenges Across 3 Notebooks

#	Challenge	Notebook	Complexity	Focus
1	CSV with Rich Metadata	1	Basic	Data handling
2	Rules-Specific HTML Metadata	1	Basic	Domain extraction
3	Domain-Specific System Prompt	1	Basic	Prompt engineering
4	Diverse Query Types	1	Basic	Testing
5	Volleyball Domain Evaluation	1	Medium	Evaluation
6	Metadata-Based Filtering	1	Medium	Retrieval
7	CSV Analyzer Tool	2	Medium	Tool creation
8	Rules Comparison Tool	2	Medium	Multi-doc reasoning
9	Multi-Tool Agent	2	Medium	Agent orchestration
10	Complex Query Handling	2	Medium	Testing
11	Agent Performance Evaluation	2	High	Evaluation
12	Advanced Statistical Tool	3	High	Data analysis
13	Volleyball Question Generation	3	High	Query expansion
14	BM25 + Vector Hybrid	3	High	Retrieval
15	Cross-Encoder Re-Ranking	3	High	Ranking
16	LLM Contextual Compression	3	High	Compression
17	Progressive Comparison	3	High	Analysis
17B	KPI-Based Agent Evaluation	3	Very High	Metrics & Performance
18	Cross-Document Reasoning	3	Very High	Synthesis
19	Intelligent Filtering	3	Very High	Dynamic retrieval
20	Production Implementation	3	Very High	Engineering

🎓 Expected Learning Outcomes

By completing all three notebooks, learners will master:

Technical Skills

- ✅ Handling diverse file formats (CSV, HTML, TXT)

- ✅ Metadata extraction and utilization
- ✅ Vector search and embeddings
- ✅ Tool creation and agent orchestration
- ✅ Advanced retrieval techniques (hybrid, re-ranking, compression)
- ✅ Production-ready error handling
- ✅ Quality metrics evaluation (groundedness, relevance)
- ✅ Performance KPI measurement and analysis

Domain Application

- ✅ Domain-specific RAG implementation
- ✅ Sports data analysis integration
- ✅ Multi-document reasoning
- ✅ Cross-format information synthesis

Advanced Concepts

- ✅ Agent reasoning and tool usage
- ✅ Progressive system enhancement
- ✅ Performance optimization
- ✅ Evaluation methodology with quantitative metrics
- ✅ Quality vs. performance trade-off analysis
- ✅ Production deployment considerations
- ✅ Data-driven agent selection strategies

📖 Implementation Tips

Best Practices

1. **Start Simple**
    - Complete Notebook 1 thoroughly before moving on
    - Understand each component before adding complexity
    - Test incremental changes
2. **Document Thoroughly**
    - Add code comments explaining reasoning
    - Document design decisions
    - Keep track of performance observations
3. **Test Extensively**
    - Create diverse test queries
    - Compare results across implementations
    - Measure performance improvements

4. **Learn from Verbose Mode**
    - Study agent reasoning patterns
    - Understand tool selection logic
    - Identify improvement opportunities
5. **Iterate and Improve**
    - Refine metadata extraction
    - Optimize chunk sizes
    - Adjust retrieval parameters
    - Enhance evaluation metrics

🔍 Evaluation Criteria

Success Metrics

Functionality (40%)

- All document types loaded correctly
- Metadata properly extracted and utilized
- Tools functioning as designed
- Agents making appropriate decisions

Performance (30%)

- Answer quality and relevance
- Retrieval precision
- System latency
- Error handling robustness

Code Quality (20%)

- Clear, commented code
- Proper error handling
- Following best practices
- Modular, reusable components

Documentation (10%)

- Detailed explanations
- Reasoning documented
- Design decisions justified
- Learning reflections included

📚 Additional Resources

Recommended Reading

- LangChain documentation on agents
- Smolagents framework guide
- ChromaDB best practices
- Cross-encoder re-ranking papers
- Hybrid search strategies

Tools and Libraries

- pandas documentation
- BeautifulSoup for HTML parsing
- sentence-transformers guide
- Transformers library docs

🚀 Next Steps After Completion

Consider these extensions:

1. **Add PDF Support** - Include PDF volleyball rulebooks
2. **Visual Data** - Add court diagrams and play illustrations
3. **Video Transcripts** - Process volleyball coaching videos
4. **Real-Time Data** - Connect to live match data APIs
5. **Multi-Language** - Support multiple languages
6. **Fine-Tuning** - Fine-tune embeddings on volleyball text
7. **Web Interface** - Build user-friendly web UI
8. **Mobile App** - Create mobile application
9. **Voice Interface** - Add voice query support
10. **Analytics Dashboard** - Build performance monitoring dashboard

---

Document Version: 1.1 **Last Updated**: March 2026 **Difficulty Level**: Intermediate to Advanced **Estimated Completion Time**: 15-20 hours across all three notebooks **Latest Update**: Added Challenge 17B - KPI-Based Agent Evaluation (Quality Metrics + Performance KPIs)

Good luck with your volleyball RAG implementation! 🏐